

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

LAUREA TRIENNALE IN INGEGNERIA INFORMATICA

Reti neurali convoluzionali per lo studio di varianti non codificanti in sequenze genomiche

LAUREANDO

Alessandro Trigolo

Matricola 2043049

RELATORE

Prof.ssa Cinzia Pizzi

Università degli Studi di Padova

ANNO ACCADEMICO
2023/2024

“ *Se fai solo quello che sai fare,*
non sarai mai più di quello che sei ora. ”

Maestro Shifu

Sommario

Le mutazioni non codificanti, aree del genoma che non vengono codificate in proteine, sono la principale causa dei disturbi genetici, tra cui le malattie mendeliane. Gli effetti funzionali di queste variazioni sono ancora difficili da comprendere completamente. Grazie a nuove tecnologie di sequenziamento — che hanno permesso un significativo arricchimento di banche di dati biologici — e allo sviluppo di supporto hardware sufficientemente potente è stato possibile progettare strumenti basati sulle reti neurali che analizzano sequenze genomiche e forniscono informazioni rilevanti legate agli effetti funzionali di queste particolari mutazioni.

Questo elaborato ha l'obiettivo di dare un'introduzione alla biologia molecolare e fornire gli strumenti matematici per comprendere le reti neurali. Più precisamente verrà esplorato il funzionamento e la struttura di una rete neurale convoluzionale con l'obiettivo di riuscire ad analizzare tre strumenti, basati su questa tecnologia. Saranno esaminati DeepSEA, Basset e DeepSATA, tre tool che hanno lo scopo di approfondire gli effetti causati dalle mutazioni non codificanti del genoma.

Abstract

Non-coding variants, genome regions that are not translated into proteins, are a major cause of genetic diseases, such as Mendelian disorders. The functional effects of these mutations remain difficult to fully comprehend. However, thanks to advances in sequencing technologies — which have greatly enriched biological data banks — and the development of sufficiently powerful hardware, it has become possible to design neural network-based tools capable of analyzing genomic sequences and providing valuable insights into the functional effects of these specific DNA regions.

This thesis aims to introduce molecular biology concepts and provide mathematical tools for understanding neural networks. Specifically, it will explore the structure and functioning of convolutional neural networks with the goal of analyzing three tools based on this technology. The thesis will focus on DeepSEA, Basset, and DeepSATA — three tools designed to enhance the understanding of the functional impact of non-coding variants.

Indice

1	Introduzione	1
2	Background biologico	3
2.1	Dogma centrale	5
2.2	Varianti non codificanti	9
3	Reti neurali	13
3.1	Principi di base ed evoluzione	14
3.1.1	Neurone artificiale	14
3.1.2	Modello percettrone	17
3.1.3	Gradient descent	18
3.1.4	Rete neurale	20
3.1.5	Backpropagation	21
3.1.6	Indici di performance	23
3.2	Reti neurali convoluzionali	26
3.2.1	Livello convoluzionale	26
3.2.2	Livello ReLU	28
3.2.3	Livello di pooling	29
3.2.4	Applicazioni	30
4	Reti convoluzionali e varianti non codificanti	31
4.1	DeepSEA	31
4.2	Basset	33
4.3	DeepSATA	34
5	Discussione	37
6	Conclusioni	41
	Bibliografia	43

Indice delle Figure

2.1	Rappresentazione schematica della cellula eucariote.	3
2.2	Rappresentazione schematica del DNA.	4
2.3	Il processo di impacchettamento del DNA.	5
2.4	Il processo di trascrizione del DNA in RNA.	6
2.5	Il processo di traduzione da mRNA a polipeptide.	7
2.6	La mitosi cellulare.	8
2.7	Il processo di replicazione del DNA.	9
3.1	Rappresentazione schematica del funzionamento di un neurone artificiale. . . .	14
3.2	Grafico della funzione gradino $1(v)$	15
3.3	Grafico della funzione segno $sign(v)$	17
3.4	Grafico della funzione sigmoide $\sigma(v)$	18
3.5	Il gradient descent in una funzione bidimensionale.	19
3.6	Rete neurale multilivello.	20
3.7	Rappresentazione della confusion matrix.	24
3.8	Illustrazione di una generica curva ROC.	25
3.9	Area sottesa alla curva ROC.	25
3.10	L'operazione di convoluzione in una matrice bidimensionale.	26
3.11	Rappresentazione di una rete neurale convoluzionale.	27
3.12	Padding di una matrice che precede la convoluzione.	28
3.13	Grafico della funzione rettificatrice $ReLU(x)$	29
3.14	Operazione di max-pooling in una matrice.	29
4.1	Rappresentazione di una PWM.	32
5.1	Confronto delle prestazioni predittive dei tre tool su diverse specie animali. . .	39

Lista degli Acronimi

AI Intelligenza artificiale (*Artificial Intelligence*)

ANN Rete neurale artificiale (*Artificial Neural Network*)

ATP Adenosintrifosfato (*Adenosine TriPhosphate*)

AUC Area sottesa alla curva (*Area Under the Curve*)

AUROC Area sottesa alla curva ROC (*Area Under the ROC Curve*)

BCE *Binary Cross Entropy*

CNN Rete neurale convoluzionale (*Convolutional Neural Network*)

DHS Sito ipersensibile alla DNase I (*DNase I hypersensitive sites*)

DL Apprendimento profondo (*Deep Learning*)

DNA Acido desossiribonucleico (*DeoxyriboNucleic Acid*)

ENCODE *Encyclopedia of DNA Elements*

FPR *False Positive Rate*

GD Discesa del gradiente (*Gradient Descent*)

GEO *Gene Expression Omnibus*

GPU Unità di elaborazione grafica (*Graphics Processing Unit*)

GWAS Studio di associazione genome-wide (*Genome-Wide Association Study*)

mRNA RNA messaggero (*messenger RNA*)

MSE Media del quadrato degli errori (*Mean Squared Errors*)

NCBI Centro nazionale per le informazioni biotecnologiche (*National Center for Biotechnology Information*)

ncDNA DNA non codificante (*non coding DNA*)

NLL *Negative Log Likelihood*

NLP Elaborazione del linguaggio naturale (*Natural Language Processing*)

NN Rete neurale (*Neural Network*)

OCB Frammento di regione aperta della cromatina (*Open Chromatin Bin*)

OCR Regione aperta della cromatina (*Open Chromatin Region*)

PDB *Protein Data Bank*

PWM *Position Weight Matrix*

ReLU Funzione rettificatrice (*Rectified Linear Unit*)

RNA Acido ribonucleico (*RiboNucleic Acid*)

ROC *Receiver Operating Characteristics*

SGD Discesa stocastica del gradiente (*Stochastic Gradient Descent*)

SRA *Sequence Read Archive*

SVM Macchine a vettori di supporto (*Support Vector Machines*)

TAD *Topologically Associating Domains*

TF Fattori di trascrizione (*Transcription Factors*)

TPR *True Positive Rate*

tRNA RNA di trasporto (*transfer RNA*)

UTR Regione non tradotta (*UnTranslated Region*)

1

Introduzione

Ad oggi l'avanzamento della genomica — ramo della biologia molecolare che si occupa di studiare il genoma degli esseri viventi — si è rivelato notevolmente significativo al fine di approfondire e comprendere malattie legate alle mutazioni del genoma degli individui. Si stima che solamente una percentuale tra l'1% e il 2% del DNA contiene i *geni*, ovvero particolari regioni che contengono tutte le informazioni necessarie per la sintesi degli aminoacidi che poi comporranno le proteine [1], [2]. Ciò nonostante, la quasi totalità dei disturbi genomici è dovuta alle mutazioni nelle regioni non codificanti [3] — dette *varianti non codificanti*. Le mutazioni in queste zone del genoma, che apparentemente svolgono funzioni marginali, sono responsabili dello sviluppo di disturbi importanti, come le *malattie mendeliane*¹, l'epilessia, malattie cardiovascolari e soprattutto tumori — tra cui il cancro del colon-retto e il tumore al seno [3]–[11]. Risulta quindi vitale continuare a studiare gli effetti che le varianti non codificanti in sequenze genomiche hanno sugli individui.

Negli ultimi decenni, il progredire delle tecniche di *sequenziamento* [12] ha dato uno slancio rilevante allo sviluppo della *bioinformatica* — disciplina che unisce informatica e biologia. La bioinformatica si interessa a organizzare dati biologici in modo tale da facilitare l'accesso e l'inserimento di nuove informazioni (come il PDB [13]), sviluppare *tool* che permettono l'analisi dei dati e infine fornire una interpretazione significativa dei risultati ottenuti [14]. Più recentemente, l'accrescimento dei dati biologici e il costante avanzamento della potenza di calcolo hanno reso possibile l'applicazione di tecniche di *deep learning* (DL) anche nel campo della bioinformatica. Questo notevole progresso consente di scoprire e perfezionare soluzioni informatiche che permettano di delineare con sempre maggior precisione il ruolo che hanno le mutazioni nelle regioni non codificanti del DNA. Grazie a queste nuove tecnologie, la *genomica funzionale* — area della genomica che si interessa a descrivere le relazioni che ci sono tra i componenti di un sistema biologico, come geni e proteine [15] — ha avuto un forte impulso nell'approfondire le varianti non codificanti, tuttavia rimangono ancora significative lacune

¹Le malattie mendeliane, causate dalla mutazione di un singolo gene, includono la fibrosi cistica e il morbo di Huntington.

nella comprensione della relazione tra mutazioni genetiche ed espressione genica. L'utilizzo di tecniche di deep learning risulta quindi cruciale per continuare la ricerca in questo ambito. L'obiettivo di questo elaborato è di discutere e confrontare tre tool che utilizzano le *reti neurali convoluzionali* per predire l'effetto delle varianti non codificanti su sequenze genomiche: DeepSEA [16], Basset [17] e DeepSATA [18].

Più precisamente, il Capitolo 2 introdurrà le basi della biologia molecolare, necessarie per comprendere interamente l'importanza delle varianti non codificanti. Successivamente, nel Capitolo 3 saranno approfonditi i principi fondamentali delle reti neurali e il modo in cui le reti convoluzionali possono essere utilizzate come ottimo strumento per predire l'effetto di sequenze genomiche. Il Capitolo 4 invece esaminerà i dettagli implementativi di ciascuno dei tre tool, indagando principalmente sugli aspetti legati alla codifica delle sequenze, alla struttura della rete e al *dataset* utilizzato per allenare il modello. Infine nel Capitolo 5 si riassumono le differenze analizzate nel capitolo precedente, offrendo una visione complessiva del confronto tra i tre tool.

Background biologico

La cellula è l'unità fondamentale della vita. La cellula è una piccola miscela acquosa con componenti chimici, racchiusi in una membrana, e possiede l'eccezionale capacità di replicarsi. Il primo elemento che permette di distinguere le cellule è la presenza di un nucleo. Vengono definite *procarioti* le cellule senza nucleo — che sono le più diffuse e compongono organismi unicellulari come i batteri e gli archei — mentre sono chiamate *eucarioti* le cellule che contengono un nucleo — le quali sono in genere più grandi e più complesse e costituiscono forme di vita multicellulari come animali, piante e funghi [19].

All'interno della cellula eucariote (Figura 2.1), immersi nel *citoplasma*, sono presenti diversi *organuli*, i quali svolgono una particolare funzione ciascuno. I *mitocondri* sono gli organuli più diffusi. Il loro compito è quello di generare energia chimica per la cellula: attraverso il processo di ossidazione di zuccheri e grassi, viene creata una sostanza che viene utilizzata nella

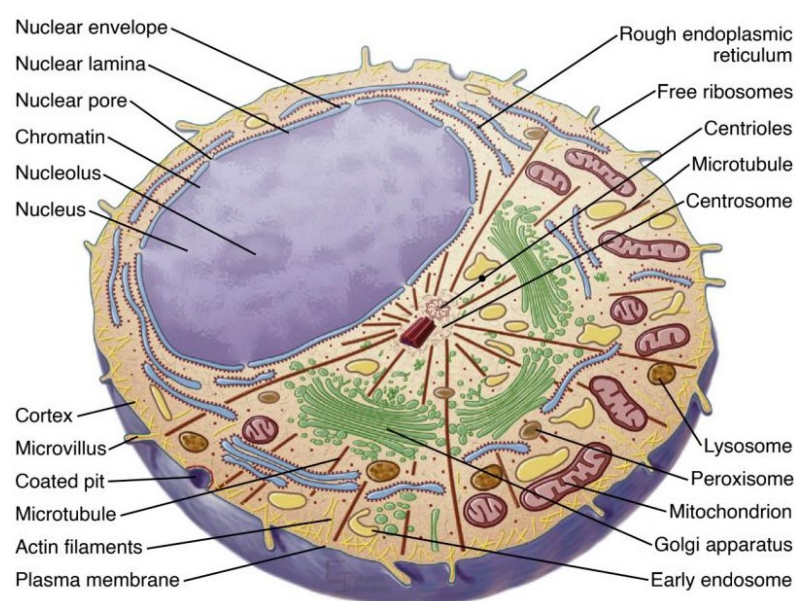


Figura 2.1: Rappresentazione schematica della cellula eucariote. Si possono notare i principali organuli tra cui i mitocondri, lisosomi e perossisomi, il reticolo endoplasmatico, e il nucleo [2].

maggior parte delle attività cellulari¹; questo processo è anche chiamato *respirazione cellulare* perché consumando l'ossigeno viene rilasciata anidride carbonica. Oltre ad essere la fonte energetica primaria della cellula, i mitocondri hanno anche importanti ruoli nella regolazione del metabolismo, del ciclo cellulare, delle risposte antivirali e anche della morte della cellula [19]–[21].

Il *reticolo endoplasmatico* è invece un organulo molto esteso e svolge molteplici funzioni. Tra questi compiti rientrano quelli di traslocazione di proteine e il ripiegamento delle proteine (*protein folding*) [19], [22]. I *lisosomi* si occupano di degradare e riciclare gli scarti cellulari e giocano un ruolo fondamentale per l'omeostasi della cellula², il suo sviluppo e il suo invecchiamento [23]–[25]. Infine, i *perossisomi* sono delle piccole vescicole che forniscono un ambiente protetto per gestire molecole tossiche come gli acidi grassi i quali sono smaltiti tramite la β -ossidazione [19], [26].

L'organulo più importante della cellula rimane il *nucleo*. Racchiuso nell'*involucro nucleare*, all'interno di questo organulo sono presenti tutte le informazioni genetiche, contenute in una lunga molecola di acido desossiribonucleico (comunemente noto come DNA), che, una volta impacchettato, forma il *cromosoma* [2], [19]. La molecola di DNA è una struttura a doppia elica formata da *nucleotidi*. Osservando la Figura 2.2, i nucleotidi sono composti a loro volta da tre elementi fondamentali: una *base azotata*, uno *zucchero* e un *gruppo fosfato*³. Le basi azotate

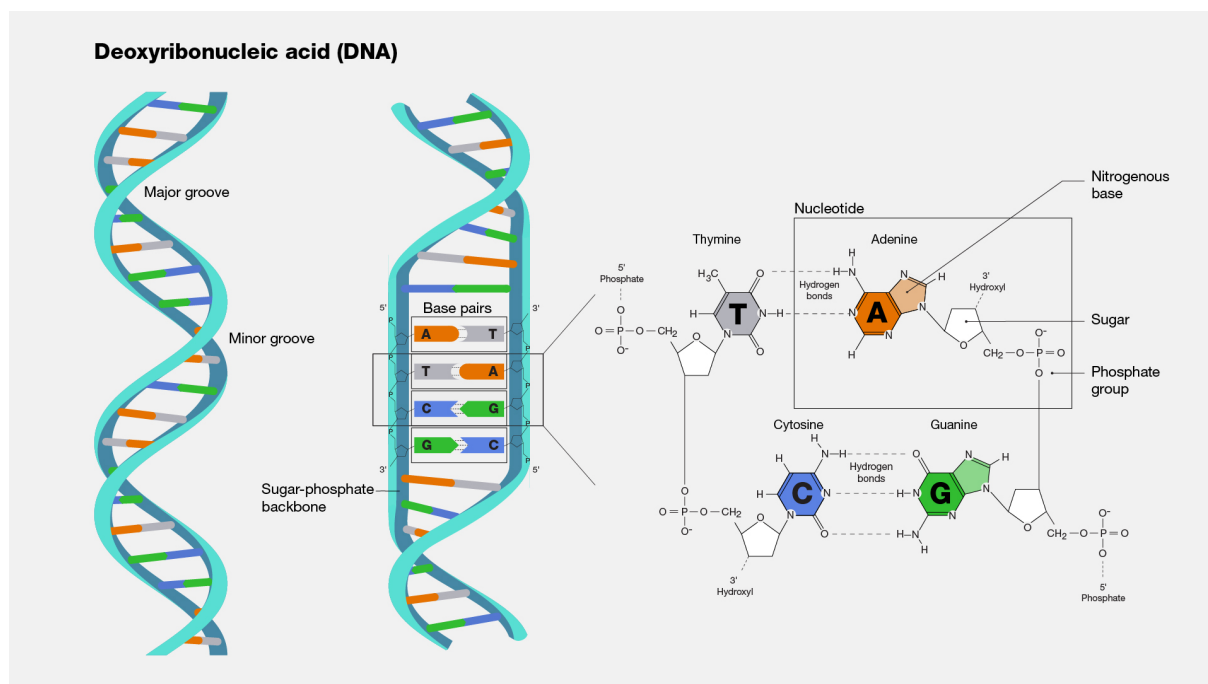


Figura 2.2: Rappresentazione schematica del DNA. Si possono osservare le coppie di basi azotate, legate tra loro attraverso gli zuccheri e i gruppi fosfati [27].

¹Questa sostanza è detta *adenosintrifosfato* o ATP ed ha una struttura simile ad un nucleotide: è infatti composta dall'Adenina, da uno zucchero e da tre gruppi fosfati.

²Con omeostasi cellulare si intende l'insieme di meccanismi necessari per mantenere ad un livello ottimale le funzioni della cellula.

³I gruppi fosfati hanno una carica negativa e forniscono alla molecola le proprietà di un acido.

sono quattro — Adenina (A), Citosina (C), Guanina (G) e Timina (T) — e si uniscono tra loro mediante dei legami ad idrogeno e secondo un preciso criterio: l'Adenina si lega solamente con la Timina (formando il legame *AT*) mentre la Citosina si unisce solo con la Guanina (creando la coppia *CG*) [1], [28]. Si osserva infine che il nucleotide di una coppia e quello successivo si legano mediante zucchero e gruppo fosfato sempre allo stesso modo: il gruppo fosfato di un nucleotide si lega sempre allo zucchero dell'altro. Di conseguenza, preso un filamento della doppia elica, le due estremità non sono uguali in quanto una termina con un gruppo fosfato (terminazione 5') e l'altra con uno zucchero (terminazione 3').

Attraverso una serie di ripiegamenti, una molecola di DNA lunga circa due metri riesce a raggomitolarsi in un cromosoma di grandezza inferiore a 2 micron (Figura 2.3). Il processo di *DNA-packaging* inizia avvolgendo la doppia elica di DNA attorno a delle proteine dette *istoni* e formando dei *nucleosomi*. In secondo luogo i nucleosomi si ammassano vicini tra loro formando una fibra, chiamata *cromatina* che, a sua volta si impacchetta su se stessa creando il cromosoma [29], [30].

2.1 DOGMA CENTRALE

La rilevanza del DNA è data dalle informazioni essenziali che questa molecola contiene. Tali informazioni risiedono nei geni, che sono delle sequenze genomiche che codificano uno o più prodotti biologici operativi [32]. L'*espressione genica* è il processo che permette di utilizzare i dati contenuti nel gene per la creazione di macromolecole, come le proteine. Per esempio, le cellule della pelle a contatto con luce solare intensa possono esprimere geni che regolano la pigmentazione della pelle [33]. L'espressione genica è divisa in due fasi principali: la *trascrizione*

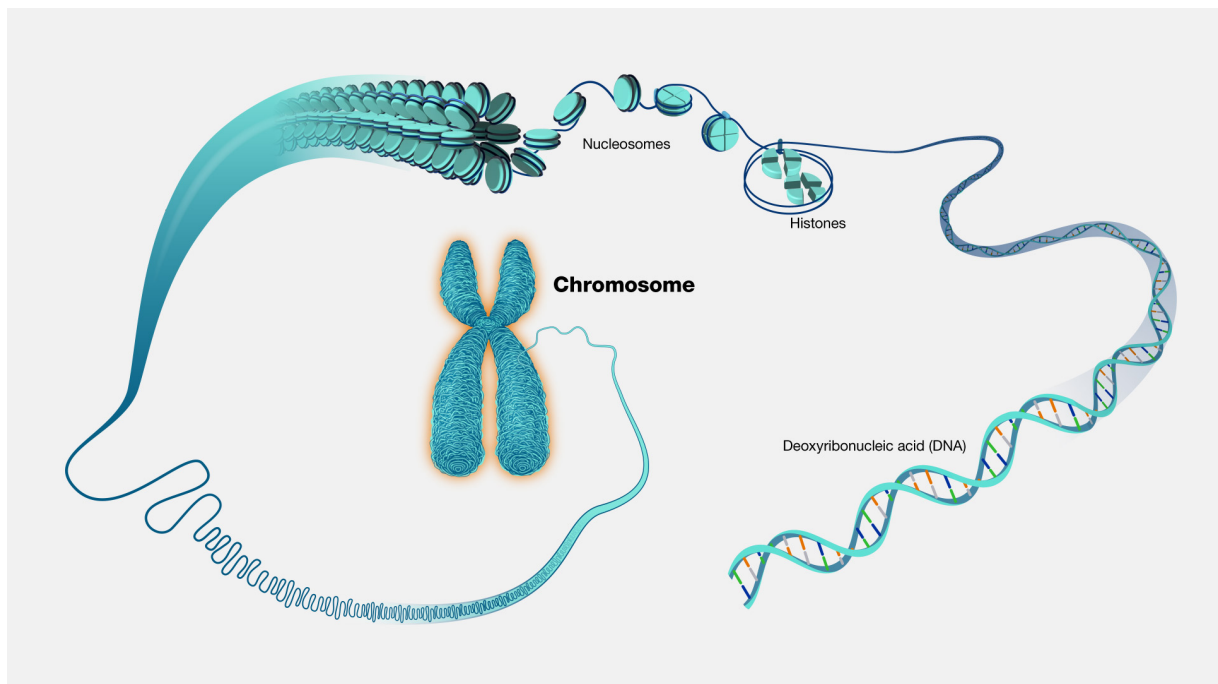


Figura 2.3: Il processo di impacchettamento del DNA che permette di compattare la struttura a doppia elica nel cromosoma [31].

zione — che si occupa di produrre delle molecole di RNA che rispecchino il gene da esprimere — e la *traduzione* — la quale traduce le informazioni dell'RNA sintetizzando la proteina.

Nella prima fase dell'espressione genica, è necessario trascrivere il DNA in una molecola molto simile ovvero l'RNA — chiamato anche acido ribonucleico. Questa molecola differisce dall'acido desossiribonucleico per una base azotata — anziché la Timina è presente l'Uracile (U) — e per lo zucchero — da desossiribosio a ribosio [34]. La trascrizione del DNA in RNA inizia quando delle proteine, chiamate *fattori di trascrizione* (TF), attratte dagli *enhancer* del DNA, riconoscono la regione che delimita l'inizio della molecola del gene da esprimere, detta *promoter*. Dopo aver riconosciuto l'inizio della sequenza, queste proteine permettono ad un enzima chiamato *RNA polimerasi* di attaccarsi ed aprire la doppia elica del DNA [35]. Una volta aperta la doppia elica, inizia la vera e propria trascrizione in RNA: il filamento del DNA viene preso come modello per la creazione dell'RNA; in particolare il nucleotide dell'RNA sarà il complementare rispetto a quello del DNA (di conseguenza $A \rightarrow U$, $C \rightarrow G$, $G \rightarrow C$ e $T \rightarrow A$). Così facendo l'acido ribonucleico viene creato un nucleotide alla volta, analizzando quello del DNA [34]. La trascrizione termina nel momento in cui gli enzimi e le proteine incontrano la regione terminatrice del gene che determina la separazione dal filamento e la terminazione dell'RNA *messenger* (mRNA) che contiene le informazioni presenti nel gene da esprimere. L'intero processo di trascrizione è illustrato nella Figura 2.4.

Prima di uscire dal nucleo l'RNA messenger subisce una serie di elaborazioni necessarie per rendere le informazioni immagazzinate sicure: diverse sono le malattie che emergono per mutazioni presenti nell'mRNA tra cui la distrofia miotonica⁴ [37]. La prima elaborazione viene

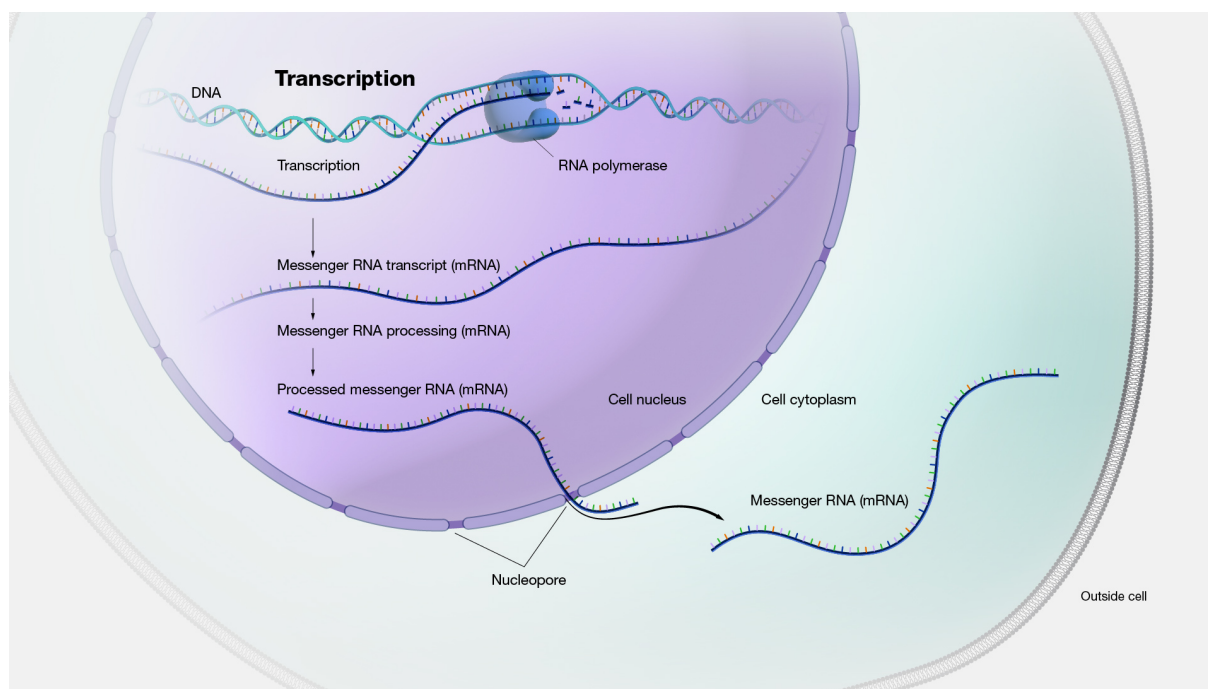


Figura 2.4: Il processo di trascrizione del DNA del gene in RNA mediante la RNA polimerasi [36].

⁴Le distrofie miotoniche sono patologie che colpiscono principalmente l'apparato muscolo-scheletrico.

chiamata *5'-end capping* e si occupa di aggiungere alla terminazione 5' dell'mRNA una Guanine attraverso un collegamento inusuale che garantisce maggiore stabilità alla molecola. In secondo luogo avviene lo *splicing* che si occupa di rimuovere le zone non codificanti — dette *intron*i — dal gene trascritto mantenendo solo quelle che verranno utilizzate per essere sintetizzate in proteine — gli *esoni* — e quindi facilitando il processo di traduzione. Infine con il *3'-end processing* viene aggiunta alla terminazione 3' dell'mRNA una coda di Adenine — detta anche *polyA tail* — che, in maniera molto simile al *5'-end capping* garantisce una stabilità del filamento di acido ribonucleico [38], [39].

Dopo essere uscito dal nucleo attraverso i *pori*, l'RNA messaggero raggiunge il citoplasma ed è pronto per iniziare la seconda fase dell'espressione genica, la traduzione. In questa fase l'mRNA viene tradotto in un *polipeptide*, ovvero una sequenza di aminoacidi che compongono la proteina. Gli aminoacidi sono più di 20, di conseguenza anziché codificare un solo nucleotide dell'RNA messaggero, vengono codificati tre nucleotidi alla volta: questa tripletta viene chiamata *codone*. Durante la fase della traduzione, giocano un ruolo fondamentale i *ribosomi* i quali sono degli organuli nei quali avviene la traduzione. I ribosomi sono composti da due sotto unità, ciascuna delle quali ha tre siti per l'RNA di *trasporto* (tRNA). Delle due sotto unità del ribosoma, quella dimensionalmente minore si lega all'mRNA e agli *anticodon*i (sequenze specifiche di tre basi nel tRNA) e controlla che la traduzione avvenga con successo. La sotto unità più voluminosa invece si prende carico di catalizzare il legame peptidico tra l'aminoacido trasportato dal tRNA e la catena di aminoacidi in crescita [39]–[41]. In questo modo i ribosomi, analizzando codone dopo codone riescono a creare la catena polipeptidica mediante l'RNA di trasporto, come mostrato nella Figura 2.5.

Una volta creata la sequenza polipeptidica, inizia il processo di ripiegamento della proteina.

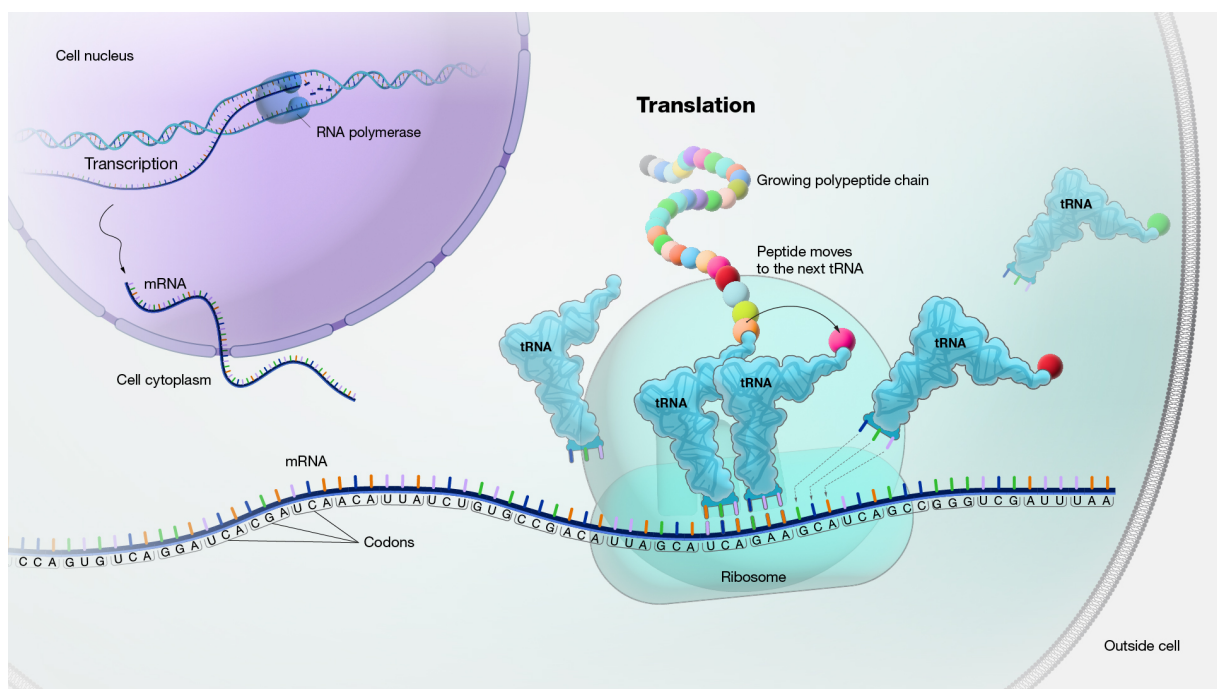


Figura 2.5: Il processo di traduzione da RNA messaggero a polipeptide attraverso il tRNA e i ribosomi [42].

In maniera molto simile a quanto visto per l'impacchettamento del DNA nel cromosoma, la sequenza di polipeptidi inizialmente si arrotola creando delle bobine che sono comunemente chiamate α -helix. Queste ultime poi si ripiegano nuovamente arrivando alla struttura terziaria della proteina, ovvero la proteina tridimensionale effettiva [43]. Una volta creata la proteina il gene è stato espresso definitivamente. Questo passaggio di informazioni dal DNA alla creazione della proteina è gergo definito come il *dogma della biologia molecolare*.

Come accennato all'inizio del capitolo, la cellula possiede la notevole capacità di replicarsi. In genere una cellula si duplica durante la crescita e lo sviluppo dell'organismo, quando deve essere rimpiazzata o rigenerata oppure nella riproduzione asessuata di alcuni microrganismi [44]. Il processo replicazione cellulare, chiamato *mitosi*, è preceduto dall'*interfase*, processo fondamentale in cui la cellula cresce di dimensioni e il DNA nei cromosomi si duplica, favorendo la replicazione cellulare. La mitosi può essere suddivisa in quattro fasi principali [44]–[47] le quali sono riassunte anche nella Figura 2.6:

1. Nella *profase* i cromosomi duplicati si condensano nel nucleo ed iniziano ad avvicinarsi dei microtuboli al nucleo, chiamati *centrosomi*; allo stesso tempo la membrana nucleare inizia a svanire;
2. Dopo che i microtuboli si sono attaccati ai cromosomi (fase intermedia detta *prometafase*) si giunge alla *metafase*, situazione in cui tutti i cromosomi sono allineati lungo la linea equatoriale della cellula;
3. Durante l'*anafase*, ciascuna coppia di cromosomi si divide e raggiunge i poli della cellula;
4. La fase finale della mitosi è la *telofase* nella quale le due cellule si dividono; le membrane nucleari delle due cellule si riformano attorno ai cromosomi divisi.

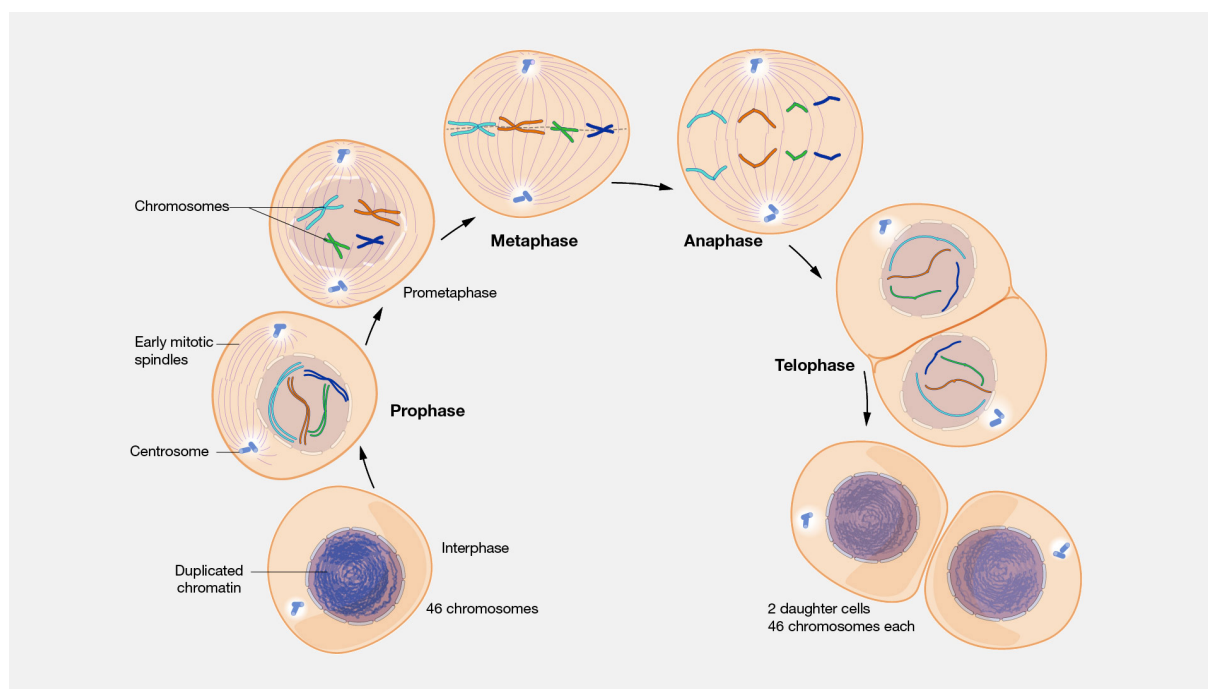


Figura 2.6: La mitosi cellulare. Si possono osservare le quattro fasi che la compongono: profase, metafase, anafase e telofase [48].

Affinché la mitosi abbia successo, è prima necessario duplicare il DNA all'interno della cellula. Il processo di replicazione di DNA che precede la mitosi è anche definito come *fase di sintesi* — *S-phase*. La duplicazione del DNA inizia con l'identificazione dell'*origine della replicazione*, ovvero una sequenza del DNA che specifica da quale punto della sequenza il DNA deve essere replicato (ci sono più di cento mila siti che segnalano un punto di origine nel DNA di una cellula). Una *proteina iniziatrice* è legata al punto di origine promuovendo l'attaccamento al DNA del *replisoma* che è composto da un enzima chiamato *elicasi* che si occupa di dividere i due filamenti di DNA procedendo nella direzione $5' \rightarrow 3'$. A questo punto il *RNA prime* inizia la sintesi del DNA favorendo l'attaccamento della *DNA polimerasi* entrambi i filamenti per duplicare il DNA. Essendo che il genoma è complementare, un filamento avrà un verso $5' \rightarrow 3'$ (*leading strand*) mentre l'altro filamento avrà verso opposto, $3' \rightarrow 5'$ (*lagging strand*). Di conseguenza, nel filamento concorde al replisoma, la polimerasi non incontrerà problemi nella duplicazione, invece nel filamento $3' \rightarrow 5'$ il DNA dovrà essere duplicato a segmenti, detti *frammenti di Okazaki* che verranno collegati tramite la *DNA ligasi* [49]–[52]. La Figura 2.7 racchiude quanto descritto sulla fase di sintesi.

2.2 VARIANTI NON CODIFICANTI

Come descritto fino ad ora, il ruolo del DNA è fondamentale in quanto trasmesso da cellula a cellula durante la replicazione per poi essere utilizzato nell'espressione genica creando le proteine. Alle regioni del DNA che prendono parte all'espressione genica, si contrappongono le regioni che non vengono codificate, come gli enhancers ed i promoters, ma anche gli introni di un gene. Le regioni non codificanti del DNA rappresentano tra il 98% e il 99% dell'intera mole-

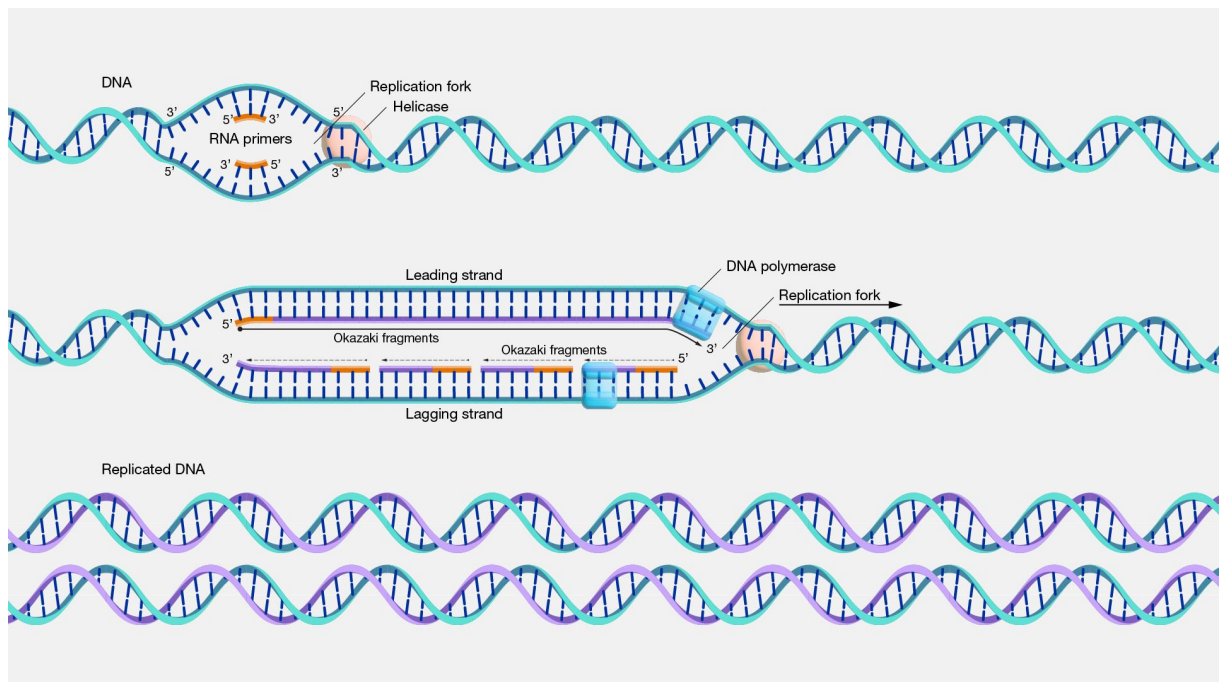


Figura 2.7: Il processo di replicazione del DNA durante la fase di sintesi [53].

cola: si pensava che queste zone non avessero una funzione effettiva, tanto che vennero definite *junk DNA*, ovvero DNA spazzatura. Contrariamente a quanto si supposeva, una mutazione in queste zone può causare diversi disturbi genetici in quanto l'organizzazione della cromatina, il processo di replicazione del DNA e l'espressione genica possono essere compromessi. Nel comprendere l'associazione tra varianti genomiche e disturbi da esse causati, il *Genome-Wide Association Study* (GWAS) ha svolto un ruolo rilevante, analizzando genomi di numerosi soggetti e correlando la presenza di una mutazione e un disturbo. Tra le varianti identificate dal GWAS, diverse sono mutazioni non codificanti [3], [54], [55].

Alcune varianti non codificanti possono alterare lo splicing di un gene trascritto. Come accennato precedentemente, lo splicing è il processo che permette la rimozione delle zone non codificanti del gene (gli introni) prima che questo sia tradotto in una proteina. Lo splicing fa parte dei processi che l'mRNA subisce prima di lasciare il nucleo ed iniziare la traduzione. All'interno degli introni sono presenti diverse sequenze che sottolineano l'inizio di un esone, tra cui il *donor*, l'*acceptor*, i *branch points* e i *tratti polipirimidinici*. Le variazioni in queste regioni possono condurre alla non codifica di un esone o alla conservazione di un introne: più del 15% dei disturbi ereditari sono dovuti proprio a questi inconvenienti. Oltre agli introni, anche altre regioni non tradotte (UTR) dell'mRNA possono condurre ugualmente a disturbi ereditari. Queste regioni sono fondamentali per gestire il processo seguente alla trascrizione: si occupano infatti di rendere il filamento di mRNA più stabile e robusto e rendono la sua localizzazione più semplice per poter iniziare il processo di traduzione [4], [8].

Le mutazioni non codificanti, oltre a verificarsi nell'RNA messaggero, possono anche occorrere nelle regioni non codificanti del DNA (ncDNA), come i promoters i quali attirano i fattori di trascrizione (TF). Queste proteine sono responsabili di agganciare alla molecola di DNA la RNA polimerasi, che si occupa di sciogliere la doppia elica ed inizia la trascrizione del gene. La variazione di queste importanti regioni rende la trascrizione del gene incorretta, conducendo ad un'erronea traduzione in proteina. Le variazioni dei promoter sono causa di malattie mendeliane e di alcuni tipi di tumori. Per lo stesso motivo, anche le mutazioni in altre sequenze *cis-regolatorie* del DNA — come enhancers e *silencers*⁵ — conducono a malattie ereditarie. Tra questi disturbi, oltre a quelli citati precedentemente si aggiungono l'aritmia cardiaca, la sindrome della gamba senza riposo⁶ e agenesia pancreatica⁷ [3], [4], [6], [8].

In aggiunta alle variazioni di enhancers e promoters del DNA non codificante, le mutazioni possono causare anche l'espansione dei *tandem repeats*, sia negli esoni, che in regioni non codificanti del DNA. I tandem repeats sono delle lunghe ripetizioni di ridotte sequenze di DNA. Le conseguenze delle variazioni dei repeat codificanti sono ben note (tra cui il morbo di Huntington), contrariamente all'effetto dell'espansione dei repeat non codificanti. A queste mutazioni si associano disturbi nella fase di trascrizione del DNA e l'intrappolamento delle

⁵A differenza degli enhancer, che stimolano una particolare trascrizione genica, lo scopo dei silencers è quello di ridurre o inibire la trascrizione di un particolare gene

⁶Questa sindrome neurologica suscita al soggetto una insopportabile necessità di muovere gli arti inferiori.

⁷Questo disturbo determina l'assenza di massa pancreatica sin dalla nascita.

proteine coinvolte durante lo splicing [4].

Le mutazioni del ncDNA, possono anche introdurre dei cambiamenti strutturali della cromatina, fibra che costituisce i cromosomi. In particolare, la cromatina, mentre si impacchetta per formare il cromosoma (Figura 2.3), forma delle particolari regioni, dette TAD (*Topologically Associating Domains*), le quali contengono gruppi di geni che interagiscono frequentemente tra loro. Ne consegue che anche una piccola variazione strutturale può condurre ad una espressione genica imperfetta, causata da una scorretta interazione tra enhancers e promoters. Si è osservato che la variazione strutturale delle TAD, oltre a causare disturbi mendeliani, può portare a diverse malattie neurologiche [4], [56]. Oltre alle TAD, anche altre regioni della cromatina hanno un ruolo importante durante la trascrizione, come le regioni ipersensibili alla *DNase I*⁸ (dette DHS, *DNase I hypersensitive sites*), i siti di legame dei fattori di trascrizione (TF *binding sites*) e le modifiche istoniche. Queste regioni fanno parte, più in generale delle regioni aperte di cromatina (OCR, *Open Chromatin Region*) le quali mutazioni possono portare ad un'espressione genica inesatta e, conseguentemente, condurre a disturbi genetici [16], [18].

Contrariamente alle mutazioni nel DNA codificante, gli effetti delle variazioni nel ncDNA sono tuttora poco comprese. Questo è dovuto alla difficoltà di determinare se una variante non codificante influenzi il *fenotipo* — ovvero l'insieme delle caratteristiche strutturali e funzionali di un individuo. Diversi sono i tool bioinformatici che sono stati sviluppati per riconoscere se una mutazione non codificante abbia un effetto funzionale, ma, poiché tali funzioni dipendono anche dal contesto specifico della sequenza le previsioni possono rivelarsi imprecise, fornendo quindi informazioni non del tutto chiare sull'effetto fenotipico della mutazione [57], [58].

⁸È un enzima che scinde il DNA in frammenti più piccoli. Spesso è utilizzato per identificare regioni di cromatina che sono più accessibili alla trascrizione.



Reti neurali

Il primo modello di intelligenza artificiale (AI) risale al 1943, dove E. McCulloch e W. Pitts cercarono di modellare un neurone come una semplice funzione predefinita. In questo modello, il neurone, generava un valore in output nel caso in cui le variabili booleane di input, una volta elaborate, superavano una soglia prestabilita [59]. Poco dopo, nel 1950, Alan Turing, pubblicò un articolo che definiva una metodologia per testare l'intelligenza di un modello [60]. Questo test — noto anche come *imitation game* — consisteva nel valutare se una macchina potesse imitare l'intelligenza umana stabilendo così un obiettivo per il campo dell'intelligenza artificiale, termine che venne coniato per la prima volta nella conferenza di Dartmouth nel 1956.

Dopo due anni, nel 1958, lo psicologo F. Rosenblatt introdusse il *perceptrone* che, a differenza del modello del '43, processava input non booleani e disponeva di pesi per bilanciare l'output [61]. Anche se il perceptrone sarà alla base delle reti neurali artificiali moderne, nei dieci anni successivi alla pubblicazione dell'articolo, le aspettative iniziali non vennero soddisfatte. Nel 1968 venne pubblicato un libro il quale analizzava le prestazioni del perceptrone e constataba le forti limitazioni del modello, come l'impossibilità di risolvere problemi non linearmente separabili [62]. In seguito ad un secondo articolo del 1973, dove si evidenziavano gli scarsi risultati ottenuti in paragone con le grandi aspettative, iniziò il *Primo Inverno dell'Intelligenza Artificiale* dove fino alla metà degli anni Ottanta molte organizzazioni governative smisero di finanziare la ricerca sull'intelligenza artificiale. L'Inverno della AI terminò nel 1985 con l'introduzione del *Gradient Descent Optimization*, algoritmo che permetteva di aggiornare i pesi in modo tale da minimizzare l'errore in una rete. Un anno dopo venne introdotto l'algoritmo della *back-propagation*, fondamentale per lo sviluppo di reti neurali, costituite da più livelli di neuroni, ciascuno dei quali è collegato al livello successivo [63].

Nonostante il grande sviluppo nella parte algoritmica, l'hardware non era computazionalmente adeguato da supportare le richieste di calcolo delle reti neurali artificiali. Questa carenza nella potenza di calcolo portò al *Secondo Inverno dell'Intelligenza Artificiale*, periodo in cui l'interesse scientifico si spostò su modelli che richiedevano meno potenza di calcolo, come le *Support Vector Machines* (SVM) introdotte nel 1963. Il Secondo Inverno della AI termi-

nò a metà degli anni Novanta quando il progresso dell'hardware riuscì a soddisfare i requisiti computazionali dei modelli basati su reti neurali. Il costante sviluppo culminò nell'ultimo ventennio quando venne introdotta la GPU, che, insieme all'aumento dei dati disponibili, accelerò notevolmente i progressi nel campo dell'AI [64], [65].

3.1 PRINCIPI DI BASE ED EVOLUZIONE

Le reti neurali artificiali (ANN) — comunemente note come reti neurali (NN) — mirano a rappresentare un modello semplificato del cervello, trattato come una struttura composta da neuroni. Risulta quindi essenziale comprendere il funzionamento del singolo *neurone artificiale* per poi esplorare la struttura di una rete neurale, che è un insieme di neuroni artificiali, collegati tra loro secondo precisi criteri

3.1.1 NEURONE ARTIFICIALE

Come un neurone biologico, il neurone artificiale (Figura 3.1) riceve un numero arbitrario n di segnali in *input*, che possono essere rappresentati con la notazione $x_1, x_2, \dots, x_n$. Tali valori possono essere raggruppati nel vettore di input $\mathbf{x} = [x_1, x_2, \dots, x_n]$. Per descrivere il risultato di tutti i segnali in ingresso del neurone, si introduce una funzione g , che generalmente è una semplice somma algebrica dei segnali di input x_i , con $i \in [1, n]$. Ogni segnale di input, prima di essere sommato viene moltiplicato per il rispettivo peso (*weight*) w_i , con $i \in [1, n]$, che appartiene al vettore dei pesi $\mathbf{w} = [w_1, w_2, \dots, w_n]$. Ne consegue che il segnale in uscita, indicato con v , comprenderà la somma del prodotto del segnale i -esimo (x_i) con il rispettivo peso i -esimo (w_i):

$$v = g(\mathbf{w}, \mathbf{x}) = \sum_{i=1}^n w_i x_i = \mathbf{w} \cdot \mathbf{x} \quad (3.1)$$

Per attivare un neurone, è necessario che il segnale v prodotto in *output* sia superiore ad una soglia prescelta, indicata con b . Questo si traduce nel sottrarre tale valore al segnale v e verificare

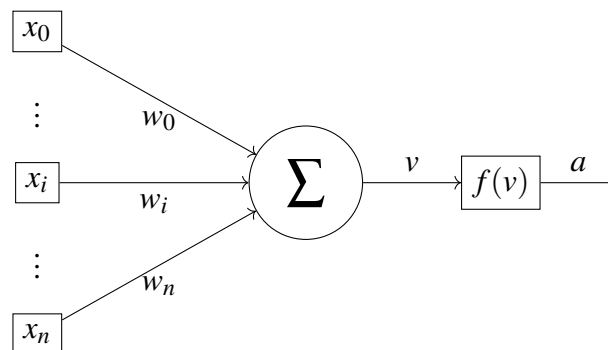


Figura 3.1: Rappresentazione schematica del funzionamento di un neurone artificiale. I segnali in input x_i vengono moltiplicati con i rispettivi pesi w_i e sommati tra loro; il risultato v viene processato dalla funzione di attivazione $f(v)$ che restituisce l'output a .

che il risultato sia maggiore o minore di zero.

$$\sum_{i=1}^n w_i x_i \geq b \quad \Rightarrow \quad \sum_{i=1}^n w_i x_i - b \geq 0$$

Il valore sottratto alla somma, detto *bias*, può essere inglobato nella somma algebrica. È sufficiente introdurre una componente aggiuntiva $x_0 = 1$ nel vettore di input $\mathbf{x}$ ed una componente $w_0 = -b$ nel vettore dei pesi $\mathbf{w}$. In questo modo, l'equazione 3.1 può essere riscritta con l'indice i che parte da 0, come segue:

$$v = g(\mathbf{w}, \mathbf{x}) = \sum_{i=0}^n w_i x_i = \mathbf{w} \cdot \mathbf{x}$$

Il criterio che descrive l'attivazione del neurone artificiale è riassunto nella *funzione di attivazione*, chiamata $a = f(v)$. La funzione di attivazione più semplice che descrive tale criterio la funzione *gradino* $\mathbf{1}(v)$, descritta graficamente nella Figura 3.2:

$$a(v) = \mathbf{1}(v) = \begin{cases} 1 & \text{se } v \geq 0 \\ 0 & \text{se } v < 0 \end{cases}$$

Analogamente alle reazioni agli stimoli esterni di un neurone biologico, anche neurone artificiale deve essere in grado di rispondere in un determinato modo quando riceve determinate informazioni. Affinché il neurone sia capace di comportarsi correttamente di fronte a specifici dati, è fondamentale allenarlo: è dunque necessario utilizzare un *training dataset* che contenga numerosi vettori di input $\mathbf{x}$ e, per ciascuno di essi sia presente la risposta corretta che il neurone dovrebbe fornire. Lavorando con numerosi vettori di input, viene introdotta la notazione X_j , che indica il j -esimo vettore $\mathbf{x}$ del dataset. Formalmente definiamo il dataset $\mathcal{D}$ come:

$$\mathcal{D} = \{ \{X_1, y_1\}, \{X_2, y_2\}, \dots, \{X_j, y_j\}, \dots, \{X_M, y_M\} \}$$

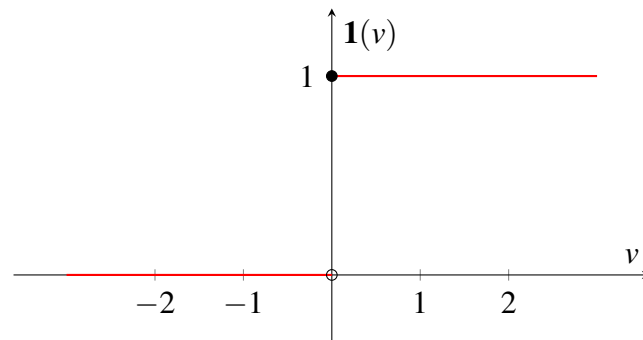


Figura 3.2: Grafico della funzione gradino $\mathbf{1}(v)$. Si osserva che vale zero per valori strettamente minori di zero e uno per valori maggiori o uguali a zero.

Il dataset¹ è composto da M vettori di input — X_j ($j \in [1, M]$) — e da esattamente M risposte y_j , che rappresentano il comportamento atteso del neurone se il vettore di ingresso è X_j . È quindi possibile definire la matrice $\mathbf{X}$ che contiene esattamente M vettori — ciascuno in una riga — i quali sono composti esattamente da n componenti (o *feature*), che sono le componenti associate al vettore dei pesi $\mathbf{w}$, precedentemente introdotto. Alla matrice $\mathbf{X}$ è associato il vettore $\mathbf{y}$, anch'esso di dimensione M tale per cui la componente y_j sia la risposta attesa del vettore di input X_j .

$$\mathbf{X} = \begin{bmatrix} X_1^0 & X_1^1 & \cdots & X_1^i & \cdots & X_1^n \\ X_2^0 & X_2^1 & \cdots & X_2^i & \cdots & X_2^n \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ X_j^0 & X_j^1 & \cdots & X_j^i & \cdots & X_j^n \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ X_M^0 & X_M^1 & \cdots & X_M^i & \cdots & X_M^n \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_j \\ \vdots \\ y_M \end{bmatrix}$$

Da questo deriva che il vettore X_j è dimensionalmente compatibile con il vettore di pesi $\mathbf{w}$ in quanto hanno esattamente la stessa dimensione.

$$X_j = [X_j^0, X_j^1, \dots, X_j^n] \quad \mathbf{w} = [w_0, w_1, \dots, w_n]$$

Perciò il dataset $\mathcal{D}$ può essere riscritto attraverso la matrice $\mathbf{X}$ ed il vettore di risposte attese associato $\mathbf{y}$.

$$\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$$

Con un dataset di partenza si può definire un algoritmo generico che sia in grado di descrivere il processo di apprendimento di neurone artificiale. Come descritto nell'Algoritmo 1, dopo aver inizializzato il vettore di pesi $\mathbf{w}$, per ogni vettore X_j del dataset $\mathcal{D}$, vengono calcolati il segnale v e il valore della funzione di attivazione $a = f(v)$. L'idea alla base dell'algoritmo è quella di modificare il vettore di pesi $\mathbf{w}$ in funzione del risultato della funzione di attivazione rispetto al segnale processato. In questo modo, una volta processati tutti i vettori presenti nel dataset, è possibile constatare se il neurone sia stato in grado di apprendere il comportamento desiderato in funzione del vettore in ingresso [64].

¹Un dataset così definito è utilizzato nel *supervised learning*, dove all'interno del dataset sono presenti sia i vettori in ingresso che la risposta attesa. Si contrappone l'*unsupervised learning* — che non verrà trattato — dove sono presenti solo i vettori in ingresso e sono sconosciute le risposte attese.

Algoritmo 1 Allenamento del neurone artificiale**Input:** Dataset $\mathcal{D}$ Inizializza $\mathbf{w}$ con numeri casuali $j \leftarrow 1$ **while** $j \leq M$ **do** Calcola v in funzione di X_j e $\mathbf{w}$ Determina a in rapporto a v e y_j Modifica $\mathbf{w}$ a seconda del risultato a $j \leftarrow j + 1$ **end while****3.1.2** MODELLO PERCETTRONE

Uno tra i modelli di neuroni artificiali utilizzati nelle reti neurali è il percettrone. Dato un vettore in ingresso X_j^2 e un vettore di pesi $\mathbf{w}$, il segnale v è dato dalla somma algebrica dei prodotti di ciascuna delle componenti:

$$v = g(\mathbf{w}, X_j) = \sum_{i=0}^n w_i X_j^i = \mathbf{w} \cdot X_j$$

La funzione di attivazione $f(v)$ del percettrone è la funzione “segno” (Figura 3.3) — chiamata anche *gradino bipolare* — definita come segue.

$$a(v) = \mathbf{sign}(v) = \mathbf{sign}(\mathbf{w} \cdot X_j) = \begin{cases} 1 & \text{se } v \geq 0 \\ -1 & \text{se } v < 0 \end{cases}$$

Il particolare più importante del percettrone è la *learning rule*, ovvero il criterio secondo il quale il vettore di pesi è aggiornato a seconda della predizione del modello. Se la predizione a_j rispetto al vettore X_j è diversa dalla risposta attesa y_j , allora il vettore di pesi è modificato come

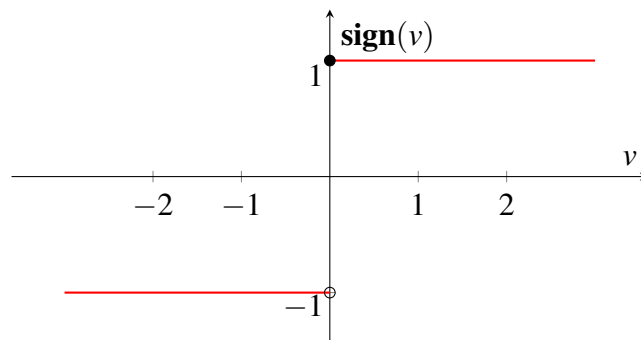
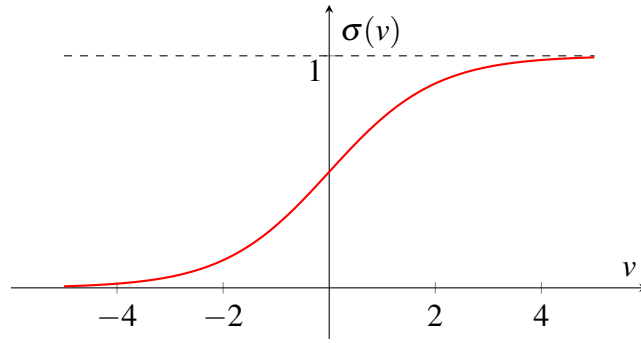


Figura 3.3: Grafico della funzione segno $\mathbf{sign}(v)$. Si osserva che vale -1 per valori strettamente minori di zero e 1 per valori maggiori o uguali a zero.

²Il vettore di input è indicato come la j -esima riga della matrice di input $\mathbf{X}$ del dataset $\mathcal{D}$.

Figura 3.4: Grafico della funzione sigmoide $\sigma(v)$.

segue [64], [66].

$$w_i = w_i + y_j X_j^i$$

Al perceptrone si aggiunge anche il *neurone sigmoideo*, che è molto simile al perceptrone se non per la funzione di attivazione. Aniché avere la funzione discontinua “segno”, viene introdotta la funzione *sigmoide* $\sigma(v)$, che è una funzione continua e restituisce valori compresi tra zero e uno (Figura 3.4). Questa funzione, definita di seguito, rimuove le discontinuità della funzione di attivazione del perceptrone e fornisce un intervallo di valori reali, piuttosto che un risultato binario.

$$\sigma(v) = \frac{1}{1 + e^{-v}} = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{X}_j}}$$

Questa importante differenza rende il neurone sigmoideo più flessibile rispetto al perceptrone classico e più adatto all’utilizzo nelle reti neurali [66].

3.1.3 GRADIENT DESCENT

La learning rule introdotta nel perceptrone, per quanto semplice, spesso viene sostituita dalla learning rule derivante dal *Gradient Descent* (GD, rappresentato nella Figura 3.5), un approccio che mira a trovare il minimo di una funzione differenziabile, spesso chiamata funzione “costo”. Fornito il vettore di pesi $\mathbf{w} = [w_0, w_1, \dots, w_n]$, durante l’allenamento del neurone, l’algoritmo cerca di modificare tale vettore minimizzando sempre di più l’errore tra il valore predetto dal modello e il risultato atteso. Più precisamente, data una funzione di costo $C(\mathbf{w})$ — detta anche *loss function* — la learning rule del GD vale:

$$\mathbf{w} = \mathbf{w} - \eta \nabla C(\mathbf{w})$$

Dove η è un parametro positivo, chiamato *learning rate* e serve per bilanciare la rapidità con cui il vettore dei pesi sia aggiornato durante l’esecuzione dell’algoritmo. Il termine $\nabla C(\mathbf{w})$ è invece il gradiente della loss function. Il gradiente di una funzione a più dimensioni è un vettore che indica la direzione di massima crescita del valore della funzione; conseguentemente

la direzione opposta è quella da seguire per raggiungere il minimo (locale o assoluto) della funzione. La Figura 3.5 mostra graficamente come l'approccio GD cerchi di raggiungere tale valore ad ogni iterazione. Si osserva che in questo caso la funzione è rappresentata in due dimensioni ma in generale la loss function è n -dimensionale. Questa funzione può essere una qualsiasi funzione errore: una tra le più comuni è la funzione che calcola la media del quadrato degli errori (in gergo chiamata MSE, *Mean Squared Errors*). Per fare un esempio pratico, avendo il vettore di pesi $\mathbf{w}$, un dataset $\mathcal{D}$ — definito come in precedenza — ed una funzione di attivazione a , la MSE è definita come:

$$C(\mathbf{w}) = \frac{1}{M} \sum_{j=1}^M [y_j - a(\mathbf{w}, X_j)]^2$$

L'approccio adottato da questo algoritmo, per quanto efficace non è completamente efficiente. La complessità computazionale richiesta è notevole: ad ogni iterazione dell'allentamento del neurone, è necessario trovare il valore della derivata della loss function, che si traduce nel computare la predizione di tutti i vettori di input X_j del dataset $\mathcal{D}$ e compararli con la risposta attesa y_j . Questo approccio, per dataset di dimensioni medio-grandi è molto poco efficiente, per quanto preciso. Inoltre è possibile che la discesa del gradiente conduca l'aggiornamento del vettore di pesi in un minimo locale della funzione errore, giungendo quindi ad una soluzione sub-ottima. Questi problemi possono entrambi essere gestiti da una variante di questo algoritmo, chiamata *Stochastic Gradient Descent* (SGD). Per ogni iterazione dell'algoritmo di allenamento, vengono presi un numero $m < M$ di vettori di input con le rispettive risposte attese

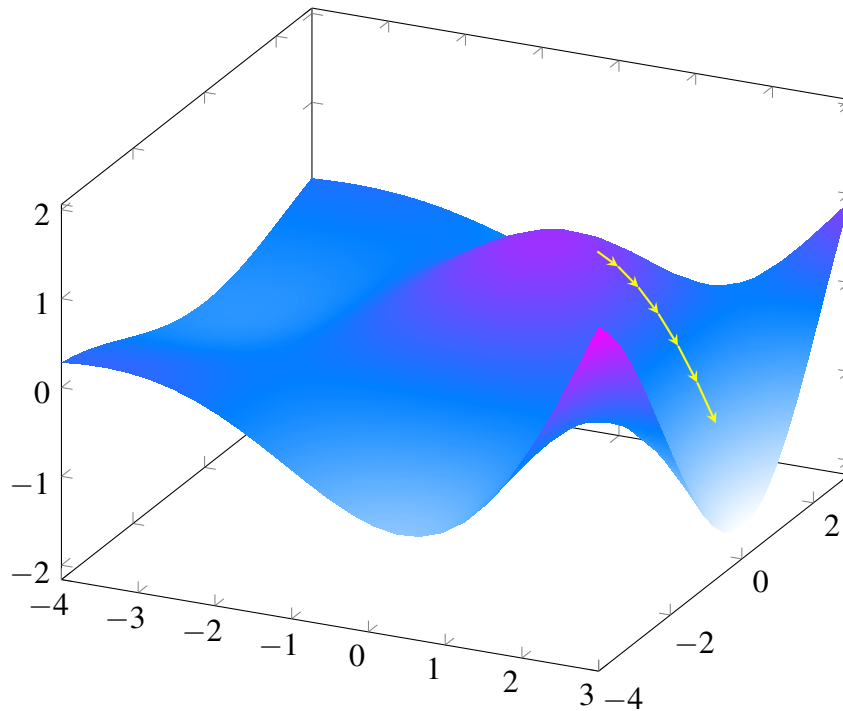


Figura 3.5: Il gradient descent in una funzione bidimensionale. Si osserva che l'algoritmo, attraverso il gradiente della funzione, riesce a spostarsi verso il minimo (locale o assoluto) della funzione, come una biglia di vetro.

e aggiornato il vettore $\mathbf{w}$ in base a questo sotto insieme del dataset, chiamato anche *mini-batch*. In questo modo la complessità computazionale viene drasticamente diminuita in quanto calcolare la derivata della loss function per una frazione ridotta del dataset richiede meno risorse. In aggiunta la casualità della scelta del mini-batch può evitare la convergenza in un minimo locale poiché rende la discesa lungo la funzione di errore leggermente più imprevedibile, diminuendo la possibilità di risultati sub-ottimi [66]–[68].

3.1.4 RETE NEURALE

Una rete neurale è una struttura composta da neuroni artificiali i quali sono organizzati in livelli (*layers*). In una rete neurale multilivello, è sempre presente un input layer e un output layer. I livelli che si trovano tra questi sono detti livelli nascosti — *hidden layers*. I neuroni dello stesso livello non sono mai collegati tra loro ma solo con i neuroni del livello precedente e del livello successivo. Più precisamente, data una rete neurale, i neuroni del livello ℓ ricevono il segnale dai neuroni del livello $\ell - 1$ e, dopo aver elaborato le informazioni, inviano il segnale processato ai neuroni del livello $\ell + 1$: si dice che la rete è di tipo *feedforward* in quanto non sono presenti cicli nella struttura. Osservando la Figura 3.6 si introduce una nuova notazione per i neuroni all'interno di una rete. Si definisce con $N_j^{(\ell)}$ il neurone j -esimo che si trova nel livello ℓ e si identifica con la notazione $a_j^{(\ell)}$ l'output della funzione di attivazione del neurone $N_j^{(\ell)}$. Il peso $w_{j,i}^{(\ell)}$ è il peso che collega il neurone $N_i^{(\ell-1)}$ al neurone $N_j^{(\ell)}$, ovvero il collegamento tra il neurone alla posizione i del livello precedente $\ell - 1$ e il neurone $N_j^{(\ell)}$. Inoltre, per ogni neurone vengono esplicitati i bias: $b_j^{(\ell)}$ indica il bias associato al j -esimo neurone nel livello ℓ . Presupponendo l'utilizzo di neuroni sigmoidei all'interno della NN l'output $a_j^{(\ell)}$ del neurone

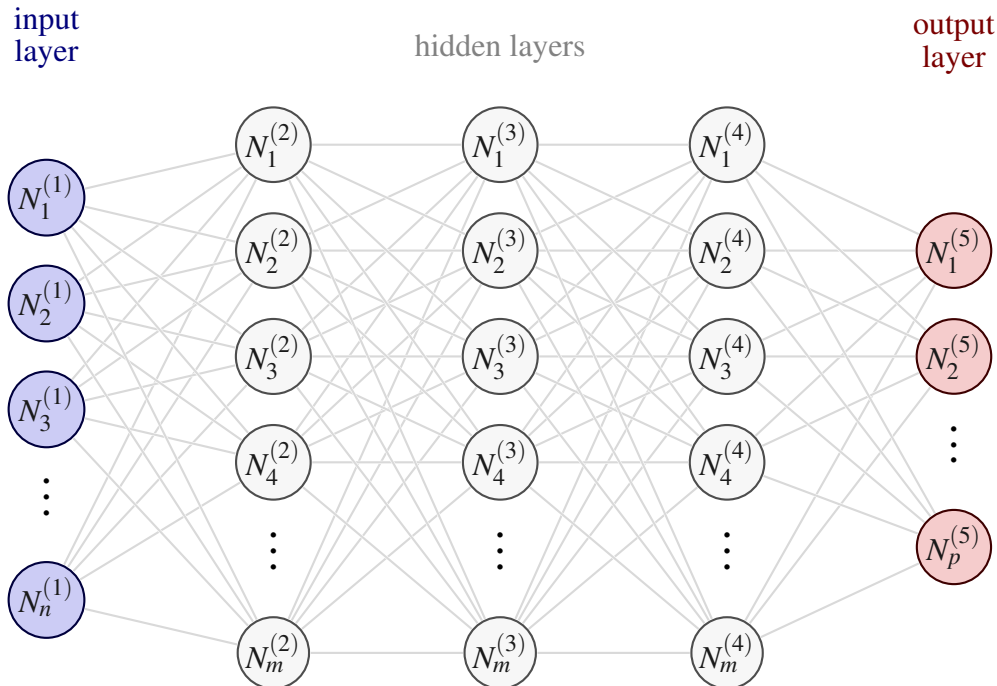


Figura 3.6: Rete neurale multilivello (immagine adattata da [69]).

$N_j^{(\ell)}$, ovvero l'output della funzione di attivazione del neurone può essere calcolato come segue:

$$a_j^{(\ell)} = \sigma(v) = \sigma\left(\sum_i w_{j,i}^{(\ell)} a_i^{(\ell-1)} + b_j^{(\ell)}\right)$$

Questa formula indica che l'output della funzione di attivazione del neurone $N_j^{(\ell)}$ è dato dalla sigmoide calcolata sull'input del neurone. L'input è dato dalla somma algebrica degli output dei neuroni del livello precedente ($\ell - 1$), moltiplicati per i rispettivi pesi e sommati al bias del neurone. Questa notazione può essere riscritta in forma matriciale per renderla più elegante. Si definisce la matrice $\mathbf{W}^{(\ell)}$ che contiene i pesi che collegano i neuroni del livello $\ell - 1$ al livello ℓ . Allo stesso modo, si può definire il vettore $\mathbf{b}^{(\ell)}$, che contiene i bias dei neuroni nel livello ℓ . Infine il vettore $\mathbf{a}^{(\ell)}$ contiene i valori di output delle funzioni di attivazione di tutti i neuroni al livello ℓ . La formula può quindi essere riscritta come segue:

$$\mathbf{a}^{(\ell)} = \sigma\left(\mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right) = \sigma\left(\mathbf{v}^{(\ell)}\right)$$

Questo risultato fornisce una visione di insieme dell'output del livello ℓ , rispetto che al singolo neurone $N_j^{(\ell)}$. Inoltre si osserva che è stato definito un vettore $\mathbf{v}^{(\ell)}$ il cui valore non è altro che l'input della funzione sigmoide, ovvero la somma pesata degli input del livello $\ell - 1$.

3.1.5 BACKPROPAGATION

Come per il caso del singolo neurone artificiale, anche una ANN deve essere allenata. Per allenare una rete neurale si utilizza l'algoritmo della *backpropagation*, introdotto per la prima volta nel 1986. La backpropagation, sfruttando i principi del GD, modifica i pesi della rete con l'obiettivo di minimizzare una *cost function* C . Questo si traduce nel calcolare il gradiente di C , in particolare la derivata parziale della funzione C rispetto al peso di un neurone e rispetto al suo bias:

$$\frac{\partial C}{\partial w_{j,i}^{(\ell)}} \quad \frac{\partial C}{\partial b_j^{(\ell)}}$$

Oltre a questo viene definito anche l'errore del neurone $N_j^{(\ell)}$, chiamato $\delta_j^{(\ell)}$ indicato come:

$$\delta_j^{(\ell)} = \frac{\partial C}{\partial v_j^{(\ell)}}$$

Questo importante valore, indica di quanto varia la cost function in rapporto ad un piccolo cambiamento nell'input del neurone $N_j^{(\ell)}$. Supponendo di essere al livello ℓ , modificare il vettore $\delta^{(\ell)}$, quindi modificare di poco il vettore di input $\mathbf{v}^{(\ell)}$, può portare alla riduzione complessiva della loss function C , propagando ai livelli successivi il cambiamento dell'input al livello ℓ . L'algoritmo della backpropagation fornisce un modo per calcolare il valore $\delta_j^{(\ell)}$ e collegarlo alle derivate parziali.

La backpropagation può essere descritta attraverso quattro equazioni fondamentali [66]. La prima equazione fornisce un modo per calcolare il valore $\delta_j^{(L)}$, ovvero l'errore di predizione nel livello di output L . Come si può osservare dall'equazione 3.2, tale valore è il prodotto di due derivate. La prima derivata parziale indica il cambiamento della cost function rispetto all'output del neurone $N_j^{(L)}$: se il neurone ha poca influenza sull costo finale, allora $\delta_j^{(L)}$ è un valore molto piccolo. La seconda derivata indica quanto la funzione sigmoide varia nel punto $v_j^{(L)}$.

$$\delta_j^{(L)} = \frac{\partial C}{\partial a_j^{(L)}} \sigma' \left(v_j^{(L)} \right) \quad (3.2)$$

La seconda equazione che descrive la backpropagation mette in relazione $\delta^{(\ell)}$ con l'errore del livello successivo $\delta^{(\ell+1)}$, come mostrato nell'equazione 3.3. Moltiplicare la matrice trasposta dei pesi $\left(\mathbf{W}^{(\ell+1)} \right)^T$ con l'errore $\delta^{(\ell+1)}$ è come, intuitivamente, misurare l'errore in uscita al livello ℓ . Questo risultato viene moltiplicato con $\sigma' \left(v^{(\ell)} \right)$ attraverso l'operatore “ $\odot$ ”, ovvero il prodotto *element-wise*³. Questo passaggio permette di propagare l'errore “indietro” attraverso la funzione di attivazione, fornendo quindi il valore di $\delta^{(\ell)}$. Questa equazione fornisce un modo per calcolare i pesi del livello precedente propagando all'indietro l'errore mediante la trasposta dei pesi e la derivata della funzione di attivazione.

$$\delta^{(\ell)} = \left[\left(\mathbf{W}^{(\ell+1)} \right)^T \delta^{(\ell+1)} \right] \odot \sigma' \left(v^{(\ell)} \right) \quad (3.3)$$

La terza equazione (3.4), mette in relazione la derivata della loss function rispetto al bias di un neurone ($b_j^{(\ell)}$) e il suo errore $\delta_j^{(\ell)}$.

$$\frac{\partial C}{\partial b_j^{(\ell)}} = \delta_j^{(\ell)} \quad (3.4)$$

Infine la quarta equazione (3.5) lega la derivata della cost function rispetto al peso di un neurone e l'errore del neurone. Più precisamente, l'errore $\delta_j^{(\ell)}$ del neurone $N_j^{(\ell)}$ è legato alla derivata della funzione di costo rispetto al peso $w_{j,i}^{(\ell)}$ mediante il valore in output $a_i^{(\ell-1)}$ del neurone i -esimo che si trova al livello precedente $\ell - 1$.

$$\frac{\partial C}{\partial w_{j,i}^{(\ell)}} = a_i^{(\ell-1)} \delta_j^{(\ell)} \quad (3.5)$$

Dopo aver definito le 4 equazioni fondamentali dell'algoritmo, è possibile comprendere lo pseudocodice della backpropagation, fornito nell'Algoritmo 2. Si osserva che questo algoritmo calcola i valori degli errori dei neuroni partendo dalla fine e propagandoli all'indietro giungendo infine a computare il gradiente della funzione di costo C .

³Dati due vettori, $a = [a_1, a_2]$ e $b = [b_1, b_2]$, allora il loro prodotto vale $a \odot b = [a_1 b_1, a_2 b_2]$

Algoritmo 2 Backpropagation

Input: insieme degli output delle funzioni di attivazione dell'input layer, $\mathbf{a}^{(1)}$

Per ogni $\ell \in [2, L]$, calcolare $\mathbf{v}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ e $\mathbf{a}^{(\ell)} = \sigma(\mathbf{v}^{(\ell)})$

Calcolare l'errore di predizione $\delta_j^{(L)} = \frac{\partial C}{\partial a_j^{(L)}} \sigma'(\mathbf{v}_j^{(L)})$

Per ogni $\ell \in [L-1, 2]$, calcolare $\delta^{(\ell)} = \left[\left(\mathbf{W}^{(\ell+1)} \right)^T \delta^{(\ell+1)} \right] \odot \sigma'(\mathbf{v}^{(\ell)})$

Output: gradiente della cost function, fornito da $\frac{\partial C}{\partial w_{j,i}^{(\ell)}} = a_i^{(\ell-1)} \delta_j^{(\ell)}$ e $\frac{\partial C}{\partial b_j^{(\ell)}} = \delta_j^{(\ell)}$

Questo fondamentale algoritmo, calcolando il gradiente della funzione costo per ogni iterazione — o, in gergo, *epoca* — dell'allenamento della rete, cerca di raggiungere il minimo della loss function C , ottimizzando i valori dei pesi. In questo modo la rete neurale artificiale è in grado di apprendere dai dati forniti in ingresso in modo tale da fornire predizioni valide a dati non presenti nel dataset. Eseguire un algoritmo così potente richiede uno sforzo computazionale notevole: le reti neurali moderne sono composte da migliaia di neuroni per livello che richiedono milioni di pesi da ottimizzare precisamente. Proprio per questo motivo, con lo sviluppo di hardware avanzato, solo nell'ultimo decennio si è riusciti ad affrontare queste sfide computazionali [64], [66], [70].

3.1.6 INDICI DI PERFORMANCE

Generalmente le reti neurali si occupano di processare un particolare input e fornire in output un valore, compreso tra 0 ed 1, che indica la probabilità che l'input appartenga ad una certa categoria, chiamata *classe*. Dopo l'allenamento della rete, è essenziale valutare quanto l'allenamento sia stato efficace, analizzando il comportamento del modello con un dataset che contiene dati mai utilizzati durante la fase di allenamento, chiamato *testing dataset*. Una volta che la rete processa il dataset di test, le predizioni rispetto ai dati possono essere divise in quattro tipologie. Se la predizione del modello è *positiva*, ovvero identifica correttamente un elemento come appartenente ad una certa categoria, e tale elemento è realmente appartenente, si ha un vero positivo, o *true positive* (TP). Analogamente se viene riconosciuto un input come non appartenente ad una classe e non lo è effettivamente, si ha un vero negativo, o *true negative* (TN). Negli altri due casi, il modello categorizza l'elemento come appartenente ad una classe quando in realtà non lo è — si ha quindi un *false positive* (FP) — oppure categorizza un elemento appartenente ad una classe quando in realtà lo è — *false negative* (FN). La *confusion matrix* (Figura 3.7) riassume esattamente questo concetto.

Essendo che l'output layer fornisce la probabilità che il dato fornito appartenga o meno

		Valore predetto	
		p	n
Valore reale	p	True Positive	False Negative
	n	False Positive	True Negative

Figura 3.7: Rappresentazione della confusion matrix.

ad una classe, è necessario impostare una soglia: se tale valore viene superato, allora il dato verrà considerato come appartenente alla categoria; al contrario, verrà escluso. Per scegliere la soglia migliore viene utilizzata la curva ROC (*Receiver Operating Characteristics*), che mette in relazione due valori: il *True Positive Rate* (TPR) con il *False Positive Rate* (FPR). Il TPR è il rapporto tra i veri positivi (ovvero i valori che sono stati classificati come positivi e lo sono realmente) e tutti i valori che in realtà sono positivi (quindi i true positive e i false negative):

$$\text{TPR} = \frac{TP}{TP + FN}$$

Un elevato TPR indica che il modello è in grado di riconoscere correttamente gli elementi positivi, ovvero quelli che appartengono alla categoria in esame. Contrariamente, un basso TPR, suggerisce che il modello non è in grado di riconoscere le istanze positive, facendo quindi aumentare il numero di falsi negativi. In contrapposizione al TPR è presente il FPR, definito come il rapporto tra i falsi positivi (ovvero gli elementi che sono negativi ma classificati come positivi) e tutti i valori negativi (ovvero i falsi positivi e i veri negativi):

$$\text{FPR} = \frac{FP}{FP + TN}$$

Questo valore indica quanto il modello è capace di riconoscere le classi negative. Più piccolo è il suo valore e più il modello è accurato nel riconoscimento. Conseguentemente, un FPR elevato indica che il modello non è capace di distinguere adeguatamente le classi negative da quelle positive [71].

La curva ROC rappresenta, per soglie diverse (comprese tra 0 ed 1), il valore del TPR e del FPR del modello. Come si può osservare nella Figura 3.8, vengono rappresentate tre casistiche. La linea retta, tratteggiata, rappresenta un modello casuale, ovvero che classifica i dati senza criterio. La linea blu rappresenta un modello generico. All'inizio della curva si può osservare che il FPR cresce lentamente rispetto al TPR, indicando che il modello riesce ad identificare molti veri positivi senza classificare molti falsi positivi. In questo caso, la soglia è molto elevata, poiché è necessario che un elemento abbia un'alta probabilità di appartenenza alla classe affi-

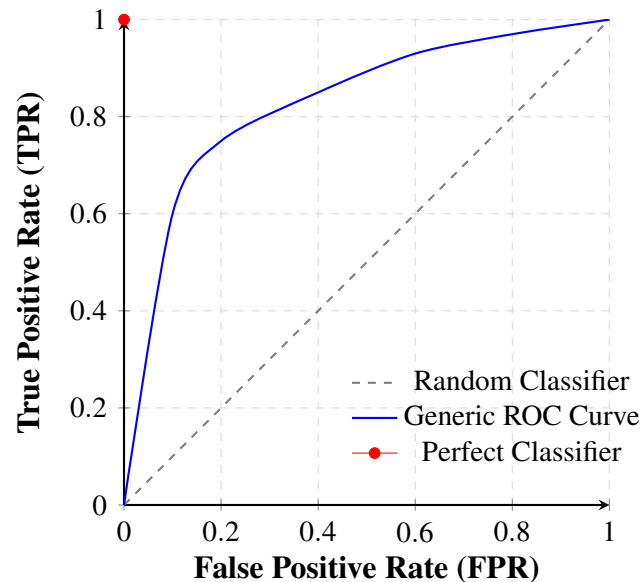


Figura 3.8: Illustrazione di una generica curva ROC.

ché venga classificato nella categoria positiva. Man mano che la soglia diminuisce si osserva che sia il TPR che il FPR del modello salgono in quanto molti più elementi vengono classificati nella categoria positiva, anche se la classe reale è quella negativa. Il punto rosso rappresenta il modello ideale, che ha il massimo valore di TPR (ovvero 1) e il minimo valore di FPR (cioè 0).

Per semplificare la lettura della curva ROC, spesso viene utilizzato il valore dell'area sotto la curva, chiamato AUC, *Area Under the Curve* — o anche AUROC, *Area Under the ROC Curve*. Questo valore aiuta notevolmente a riassumere le capacità predittive dell'algoritmo in un solo numero (Figura 3.9). È stato dimostrato che questo valore fornisce valori più significativi

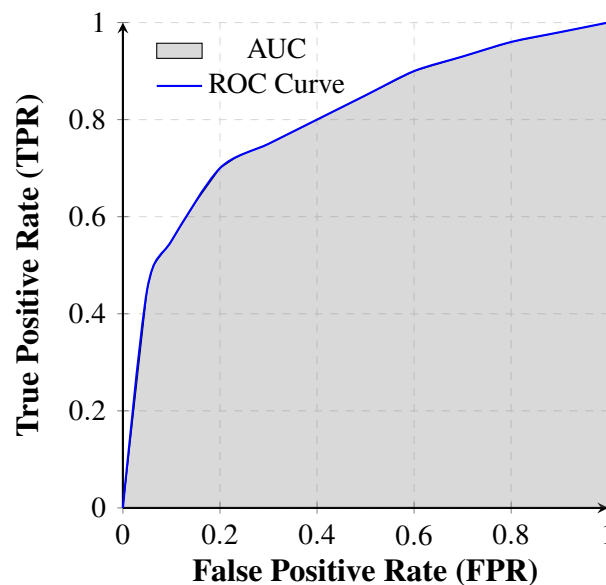


Figura 3.9: Area sottesa alla curva ROC. Questo valore viene comunemente utilizzato per paragonare diversi modelli.

rispetto ad altre metriche, tra cui l'accuratezza⁴ [72]. Inoltre questo valore è anche più comodo per paragonare le performance di due modelli, piuttosto che comparare le curve ROC: proprio per questo motivo è ad oggi una tra le metriche utilizzate per la valutazione di modelli di DL.

3.2 RETI NEURALI CONVOLUZIONALI

Le reti neurali convoluzionali (CNN) sono state introdotte per la prima volta nel 1998, con la pubblicazione dell'articolo "Gradient-based learning applied to document recognition" [73] il quale descrive una rete artificiale, chiamata *LeNet-5*, il obiettivo era quello di identificare documenti scritti a mano. Questo articolo getta le basi per la progettazione di una rete convoluzionale [74].

Una rete convoluzionale è una rete neurale feedforward che contiene dei particolari livelli, volti a ridurre il numero di pesi della rete. Tre sono i layer fondamentali che caratterizzano una CNN: *convolution layer*, *padding layer* e *ReLU layer*. Queste reti, sono organizzate in modo tale da riconoscere le caratteristiche spaziali di un'immagine. Per questo motivo, generalmente, si rappresenta l'input layer come una matrice bidimensionale⁵ di pixel: in questo modo viene anche facilitata la comprensione delle operazioni che processano l'input.

3.2.1 LIVELLO CONVOLUZIONALE

Nelle reti neurali convoluzionali, i parametri sono raggruppati in unità bidimensionali chiamati filtri o *kernel*. Questi sono delle matrici, generalmente quadrate, dimensionalmente minori rispetto alla grandezza dell'input. Tali unità rendono possibile l'operazione di convoluzione con il livello precedente. Come mostrato nella Figura 3.10, la convoluzione fa scorrere il filtro K sulla matrice M e, per ogni spostamento, calcola il prodotto scalare tra il kernel e i valori di M selezionati in quell'istante. Si osserva che il numero di valori risultanti da questa operazione

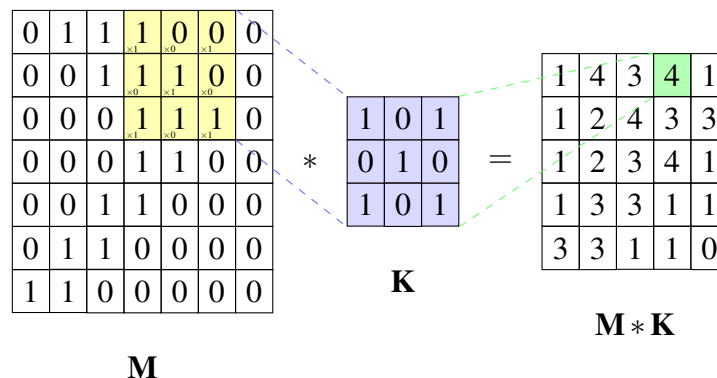


Figura 3.10: L'operazione di convoluzione in una matrice bidimensionale (adattata da [75]).

⁴L'accuratezza di un modello è data dal rapporto tra le predizioni corrette del modello (TP e TN) con l'intero dataset.

⁵Ipotizzando, per semplicità, che l'immagine sia in scala di grigi.

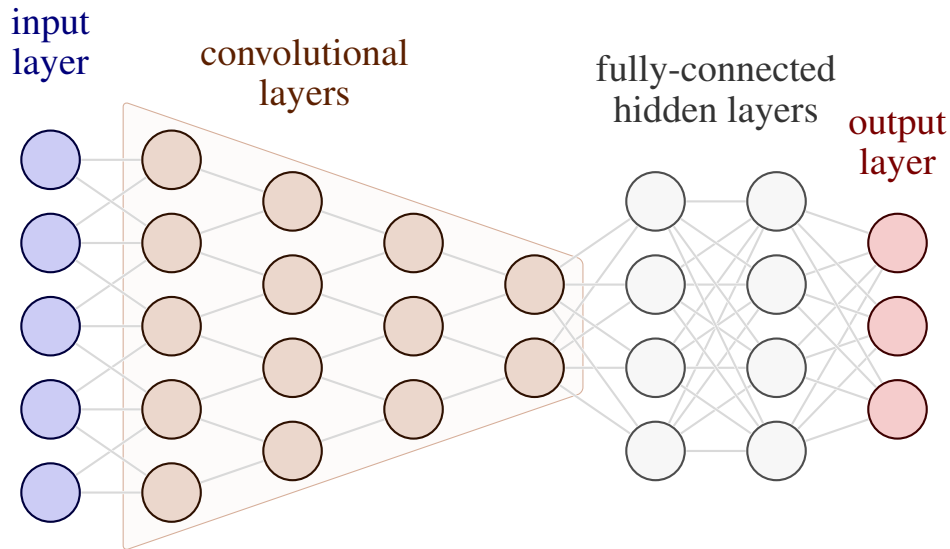


Figura 3.11: Rappresentazione di una rete neurale convoluzionale. Si fa notare la presenza di più livelli convoluzionali che permettono l'estrazione di pattern progressivamente più complessi (adattata da [69]).

è esattamente il numero di neuroni che sono presenti nel livello successivo. Questo valore si può calcolare in maniera semplice, conoscendo le dimensioni della matrice iniziale e del kernel. Fornite il numero di colonne della matrice, ovvero la sua larghezza w , e il numero di righe, ovvero la sua altezza h , allora, applicando una convoluzione con un filtro $n \times n$ le dimensioni al livello $\ell + 1$ valgono:

$$w^{(\ell+1)} = w^{(\ell)} - n + 1 \quad h^{(\ell+1)} = h^{(\ell)} - n + 1$$

Osservando la Figura 3.10 si nota che la matrice M di dimensione 7×7 diventa una matrice 5×5 quando applicata una convoluzione con un filtro di dimensione 3×3 . Si specifica inoltre che tra un livello e quello successivo, non si esegue una sola operazione di convoluzione, bensì vengono compiute più convoluzioni, ciascuna con un filtro diverso (anche se della stessa dimensione). Di conseguenza, il risultato di queste operazioni sarà un numero di matrici d , che corrispondono al numero di filtri con cui è stata attuata la convoluzione sul livello precedente. L'applicazione di più filtri sullo stesso livello rende possibile riconoscere pattern particolari: per esempio è possibile utilizzare un insieme di filtri per riconoscere una sagoma all'interno di un'immagine. In questo modo, in una CNN è possibile avere diversi livelli convoluzionali tali per cui ciascuno riconosce un particolare pattern dall'immagine iniziale sempre più complesso. Avere più livelli convoluzionali che processano l'immagine fa sì che l'ultimo livello convoluzionale esamini porzioni più ampie dell'immagine, riconoscendo caratteristiche e strutture più articolate [74], [76].

Come notato in precedenza, dopo ogni livello di convoluzione, la dimensione dell'immagine è ridotta in proporzione alla grandezza del filtro. In genere, la riduzione dimensionale comporta perdita di informazioni e va evitata. Per questo motivo, prima di fare l'operazione di convoluzione, si effettua il padding. Attraverso questa operazione, si aggiungono una serie di

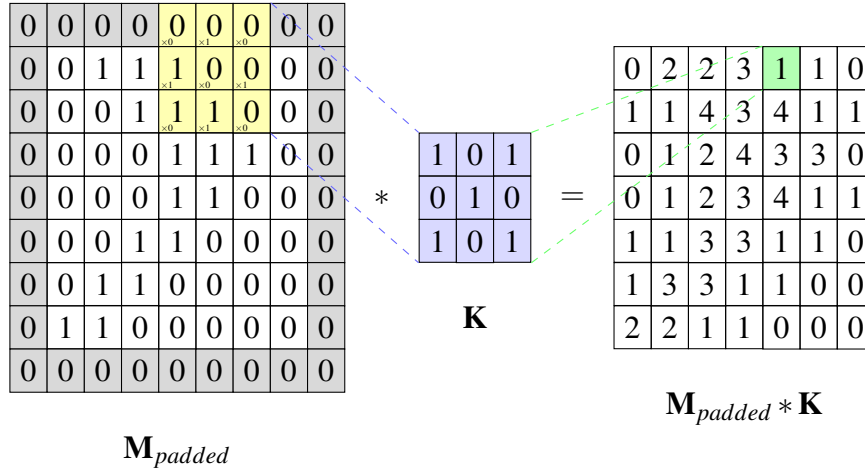


Figura 3.12: Padding di una matrice che precede la convoluzione. In questo modo, la dimensione della matrice rimane invariata dopo la convoluzione (adattata da [75]).

“0”⁶ lungo i bordi dell’immagine in modo tale che, una volta processata, rimanga della stessa dimensione. In ogni bordo dell’immagine vengono aggiunti esattamente $(n - 1)/2$ pixel, dove n è la dimensione del filtro. La Figura 3.12 illustra l’operazione di convoluzione presentata nella Figura 3.10 con la differenza che la matrice iniziale ha subito l’operazione di padding. Notando le due figure si osserva che nel primo caso il risultato della convoluzione ha ridotto la dimensione iniziale delle matrici mentre nel secondo caso, attraverso il padding, la dimensione rimane invariata grazie ai valori inseriti lungo i bordi della matrice [74].

3.2.2 LIVELLO RELU

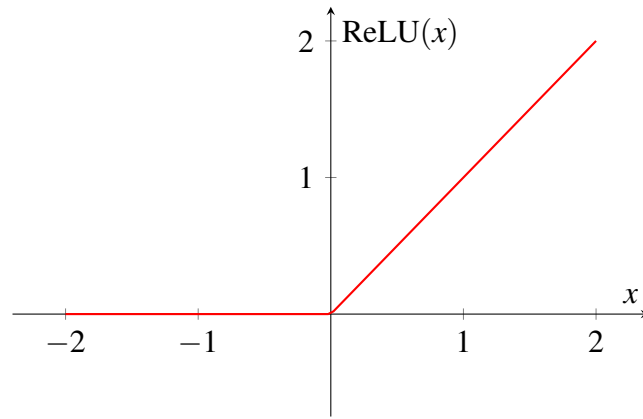
In genere, ogni livello convoluzionale è seguito da un ReLU layer (*Rectified Linear Unit*). Questo livello sostituisce la funzione di attivazione sigmoideale con una funzione detta rettificatrice, definita come segue:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{se } x > 0 \\ 0 & \text{se } x \leq 0 \end{cases}$$

Si osserva che, a differenza del livello convoluzionale, questo livello non modifica la dimensione della matrice in quanto mappa direttamente i dati presenti senza alterarne la struttura. La funzione di attivazione ReLU, rappresentata nella Figura 3.13, è stata introdotta recentemente nelle reti neurali in quanto fornisce diversi vantaggi rispetto alle funzioni sigmoideali, soprattutto per quanto riguarda la velocità dell’allenamento⁷ [74].

⁶Si è scelto il valore zero proprio perché non ha effetto e non distorce il risultato numerico della convoluzione.

⁷Questo perché la derivata di una retta è molto più semplice rispetto alla derivata della funzione sigmoide.

Figura 3.13: Grafico della funzione rettificatrice $\mathbf{ReLU}(x)$.

3.2.3 LIVELLO DI POOLING

Dopo aver processato le informazioni mediante la convoluzione, al fine di alleggerire il carico computazionale, si cerca di concentrare le informazioni estratte attraverso il *pooling*. Il tipo di pooling più comune nelle NN è il *max-pooling*. Questa operazione, in maniera simile ad un filtro, scorre lungo la matrice ed estrae dalla zona selezionata il valore massimo (Figura 3.14). L'idea dietro al max-pooling è quella di tenere le caratteristiche prevalenti dalla matrice fornita, in modo da ridurre l'informazione per rendere la fase di allenamento meno onerosa. Al max-pooling si aggiunge anche il *avg-pooling* che si occupa di calcolare la media dei pixel presenti nella porzione selezionata, estraendo quindi la “proprietà media” anziché quella prevalente [74], [77].

Infine, come in tutte le reti neurali, in una CNN sono presenti i *fully-connected layers*. Questi livelli, che sono presenti solo dopo i livelli convoluzionali (come mostrato nella Figura 3.11), hanno la stessa funzione di una classica feedforward NN: effettuare una serie di calcoli che permette di predire correttamente la risposta, in funzione dell'input fornito [74], [77].

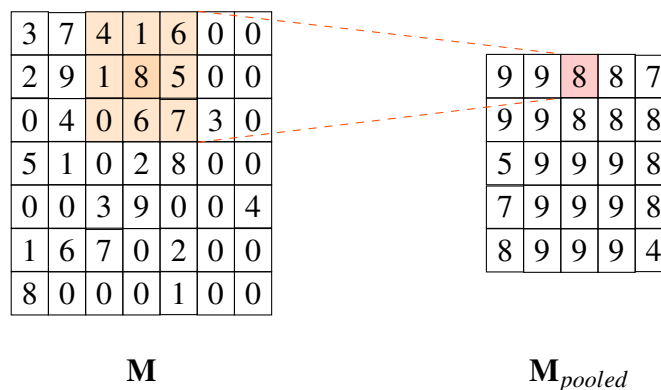


Figura 3.14: Operazione di max-pooling in una matrice. Questa operazione permette di estrarre le caratteristiche dominanti dopo la convoluzione (adattata da [75]).

3.2.4 APPLICAZIONI

Anche se fino ad ora le CNN sono state descritte come tool che processa solo immagine, il loro utilizzo è in realtà molto più vasto. Tra le loro numerose applicazioni, vengono citate il *Natural Language Processing* (NLP) — disciplina che si occupa di processare il linguaggio naturale, attraverso il riconoscimento vocale e la classificazione del testo — e *Computer Vision* — che comprende il riconoscimento visivo, la *scene labelling* e il riconoscimento di oggetti e azioni umane [78]–[80].

Oltre a quanto menzionato, le reti neurali convoluzionali, sono largamente utilizzate nel campo della bioinformatica. Più precisamente, le CNN 1-dimensionali⁸ analizzando sequenze di DNA ed RNA, apprendendo pattern — che in bioinformatica vengono anche chiamati *motif* — sequenziali man mano più complessi attraverso i diversi layer convoluzionali. In questo modo è possibile approfondire i problemi che riguardano gli elementi regolatori dell'espressione genica, tra cui le varianti non codificanti [81].

⁸Ovvero che l'input si estende in una dimensione, proprio come una sequenza.

Reti convoluzionali e varianti non codificanti

In questo capitolo verranno presentati tre tool basati su reti neurali convoluzionali, sviluppati con l'obiettivo di migliorare la comprensione funzionale delle mutazioni non codificanti. I modelli trattati sono *DeepSEA* [16], *Basset* [17] e *DeepSATA* [18]. Ciascuno di questi modelli verrà approfonditamente analizzato, in modo da comprendere le caratteristiche principali di ciascuno strumento. Più precisamente sarà discusso il dataset utilizzato per allenare il modello, la sua struttura — quindi il numero ed il tipo di livelli presenti — e le tecniche di allenamento applicate.

4.1 DEEPSEA

Il primo tool che verrà analizzato è *DeepSEA* (*Deep learning-based Sequence Analyzer*), introdotto nel 2015 con l'obiettivo di predire l'effetto delle varianti non codificanti nella cromatina. Al fine di garantire una previsione accurata, questo modello considera lunghe sequenze di DNA in modo da conoscere in maniera migliore il contesto in cui avviene la mutazione e quindi comprenderne le funzionalità, anche grazie alla struttura gerarchica dei livelli convoluzionali, che permettono di esaminare pattern locali e globali. Inoltre, *DeepSEA* è in grado di imparare contemporaneamente diverse funzionalità della cromatina utilizzando un approccio di multitask learning, che permette di addestrare il modello su più compiti correlati nello stesso momento. Per comprendere in maniera più approfondita gli effetti funzionali delle mutazioni non codificanti, è stato allenato in modo da predire 919 profili di cromatina suddivisi in tre macro categorie:

- 690 tipi di sequenze associate ai siti di binding dei fattori di trascrizione (TF), che giocano un ruolo fondamentale durante la trascrizione;
- 125 profili di regioni ipersensibili all'enzima DNasi I (DHS), che indicano la presenza o meno di elementi regolativi nel DNA, anche questi importanti durante la fase di trascrizione;
- 104 profili di modifiche istoniche, ovvero mutazioni negli istoni che rendono poco accessibile il DNA nella cromatina, impedendo quindi una corretta trascrizione;

Il modello — implementato con la libreria *Torch7* — è composto da esattamente tre livelli convoluzionali: il primo livello contiene 320 filtri, il secondo 480 e l'ultimo contiene 960 kernel. I filtri sono delle *Position Weight Matrix* (PWM), ovvero delle matrici composte da 4 righe ed M colonne che, per ciascuna base azotata, indicano la probabilità che appaia in quella determinata posizione, come mostrato nella Figura 4.1. I filtri analizzano la sequenza in input e, attraverso la convoluzione, estraggono pattern significativi, man mano spostando la finestra di una base della sequenza. Nel primo livello il kernel ha dimensione $M \times 4$, dove M è la lunghezza della finestra mentre 4 sono il numero delle basi azotate. I livelli convoluzionali successivi hanno invece dimensione $M \times k$, dove in questo caso k è il numero di kernel che sono stati utilizzati nel livello convoluzionale precedente. Dopo ogni livello convoluzionale viene applicata una ReLU (Figura 3.13) e poi viene inserito un max-pooling layer, volto ad estrarre la feature predominante dal risultato della convoluzione. In questo caso la finestra di pooling non ha uno step di uno, bensì lo step coincide con la lunghezza della finestra stessa. Infine, dopo i tre livelli convoluzionali, è presente un fully connected layer — che riceve il risultato del max-pooling layer della terza convoluzione e ci applica una ReLU — e l'output layer, che processa le informazioni con una funzione sigmoide (Figura 3.4) la quale calcola la probabilità che la sequenza in input corrisponda o meno a una tra i 919 profili.

I 919 profili di cromatina sono stati ottenuti dai progetti “*Encyclopedia of DNA Elements*” (ENCODE) e “*Roadmap Epigenomics*” [82], [83]. Le sequenze estratte sono state divise in frammenti (*bin*) da 200 bp¹, per un totale di 521636200 bp. Ciascuno di questi bin è stato poi etichettato ad uno tra i 919 profili di cromatina se almeno più della metà della sequenza corrispondeva al profilo teorico. Ogni bin estratto è stato centrato su una sequenza di 1000 bp di genoma umano, al fine di fornire a DeepSEA maggiore contesto per ricavare informazioni più significative. Ciascuna sequenza di input è stata trasformata in una matrice 1000×4 attraverso il *One-Hot encoding* che trasforma la sequenza unidimensionale in una matrice bidimensionale. Più precisamente le basi azotate, attraverso il One-Hot encoding, sono codificate come segue: $A \rightarrow [1, 0, 0, 0]$, $T \rightarrow [0, 1, 0, 0]$, $C \rightarrow [0, 0, 1, 0]$ e $G \rightarrow [0, 0, 0, 1]$. Infine alla matrice viene associata al rispettivo vettore di *label* che contiene la risposta attesa dalla rete.

	1	2	3	4	5	6	7	8	9	10
A	0.9	0.1	0.4	0.25	0.6	0.7	0.2	0.5	0.8	0.3
C	0.05	0.7	0.3	0.25	0.1	0.1	0.4	0.2	0.05	0.2
G	0.03	0.1	0.2	0.25	0.2	0.1	0.2	0.2	0.05	0.4
T	0.02	0.1	0.1	0.25	0.1	0.1	0.2	0.1	0.1	0.1

Figura 4.1: Rappresentazione di una PWM che viene utilizzata come filtro all'interno di una CNN.

¹Con “bp”, si intende *base pair*, ovvero la lunghezza della sequenza espressa nel numero di coppie di basi.

Per allenare DeepSEA in maniera efficace si è definita la cost function come la *Negative Log Likelihood* (NLL) — chiamata anche *Binary Cross Entropy* (BCE) — definita come segue:

$$\text{NLL} = - \sum_s \sum_p \log [y_{s,p} \sigma_p(X_s) + (1 - y_{s,p}) (1 - \sigma_p(X_s))]$$

In questa formula, s è l'indice del campione s -esimo (X_s) del dataset e p è l'indice del profilo della cromatina. Ne consegue che il valore $y_{s,p}$ è il valore corretto del campione s rispetto al profilo di cromatina p mentre $\sigma_p(X_s)$ è la predizione di DeepSEA sul campione X_s rispetto al profilo p . L'effettiva cost function in realtà è definita come la funzione NLL sommata ad altri valori con l'obiettivo di evitare situazioni di *overfitting*². Il gradiente della loss function è stato calcolato attraverso l'algoritmo della backpropagation, per poi essere utilizzato nell'ottimizzazione della rete. L'algoritmo di ottimizzazione utilizzato è stato il SGD con momento, che è una variante utilizzata per aumentare ancora di più le possibilità di non cadere nei minimi locali [74]. Inoltre si è scelto di applicare il *dropout training* il quale implica di annullare l'effetto di alcuni neuroni durante le epoche dell'allenamento al fine di rendere la rete più robusta al rischio di overfitting [66].

4.2 BASSET

Basset³ è un potente strumento sviluppato nel 2016, progettato per analizzare sequenze di DNA e prevedere l'accessibilità di 164 siti ipersensibili alla DNase I (DHS), che indicano la presenza di elementi regolativi. Mediante le CNN, questo modello è quindi in grado di fornire importanti informazioni riguardo la fase di trascrizione da DNA ad RNA analizzando questi particolari siti della cromatina. Proprio come DeepSEA, la presenza di più livelli convoluzionali facilita la comprensione degli aspetti funzionali derivanti dalle mutazioni nei DHS.

Anche Basset, come DeepSEA, è stato implementato utilizzando la libreria *Torch7*. Questo tool dà la possibilità di personalizzare il modello a piacimento, indicando il numero per ciascun tipo di livello, il numero di filtri (PWM) nei livelli convoluzionali, la loro dimensione, la grandezza delle finestre di pooling, il numero di neuroni nei fully-connected layer e vari iperparametri per la fase di allenamento e di test. Attraverso l'ottimizzazione Bayesiana si sono trovati i layer e gli iperparametri ideali per l'architettura. La struttura è composta da tre livelli convoluzionali: il primo livello contiene 300 filtri di lunghezza 19 bp, il secondo è composto da 200 kernel di lunghezza 11 bp e il terzo layer ne contiene 200 di lunghezza 7 bp. È importante sottolineare che dopo ciascun livello convoluzionale, segue un livello che normalizza l'output della convoluzione (*batch normalization*) seguito da una ReLU ed un max-pooling layer. In seguito ai tre livelli convoluzionali sono presenti tre fully connected layer, alternati con due

²L'overfitting è la situazione in cui il modello è completamente allineato con il dataset di allenamento, diventando quindi meno accurato nella predizioni di dati che non sono presenti nel dataset.

³Il nome richiama il bassotto, noto per le sue capacità olfattive, in analogia con l'abilità del modello di riconoscere pattern.

ReLU e due dropout layer (con coefficiente di 0.3) che, in maniera molto simile a DeepSEA sono utilizzati per prevenire il rischio di overfitting, annullando il contributo di alcuni neuroni casuali durante l'allenamento. Infine l'output layer, attraverso la funzione sigmoide, calcola la probabilità che l'input appartenga ad uno dei 164 DHS.

Dei siti ipersensibili alla DNasi I, 125 sono stati estratti da ENCODE e 39 “*Roadmap Epigenomics*”. I dati estratti sono stati processati e tutti i siti sono stati isolati ed arricchiti fino ad avere un dataset iniziale composto da sequenze di 600 bp, per un totale di 2071 886 bp. Ciascuna delle sequenze presenti nel dataset sono state poi associate al vettore di label che indicava a quale dei 164 tipi la sequenza apparteneva. Dei 164 tipi, il 17% dei siti sono stati associati a promoters, il 47% è stato classificato come siti intragenici — ovvero all'interno dei geni — e il restante 36% è stato etichettato come siti intergenici — cioè tra i geni. Infine, prima di essere utilizzati nella rete, i siti sono stati processati mediante il One-Hot encoding, formando quindi sequenze di input di dimensione 600×4 . Del dataset totale, 71 886 basi sono state utilizzate per la fase di test e altre 70 000 per la fase di validazione.

Il modello è stato allenato attraverso il GD stocastico, cercando di ottimizzare la funzione BCE il cui gradiente è stato calcolato mediante la backpropagation. Per prevenire il rischio di overfitting si è applicata la tecnica dell'*early stopping*, facendo terminare l'allenamento dopo 12 epoche che la *validation loss* rimane invariata.

4.3 DEEPSATA

Pubblicato nel 2023, DeepSATA è il terzo tool basato su una CNN che si occupa di identificare le OCR — regioni aperte della cromatina — cercando di comprenderne anche la funzione. In particolare DeepSATA si concentra sulle mutazioni non codificanti nei siti di TF binding non solo in sequenze genomiche umane, ma anche di altre specie animali quali maiali, galline, bovini e topi.

DeepSATA è un modello basato su DeepSEA. È quindi composto da tre layer convoluzionali, con rispettivamente da 320, 480 e 960 kernel (anche in questo caso rappresentati da PWM). Ogni livello convoluzionale è seguito da un layer che applica la funzione ReLU e poi un max pooling layer estrae dal risultato le feature predominanti. Dopo ciascun livello convoluzionale è presente anche un dropout layer che, come per Basset, aiuta la rete a prevenire il rischio di overfitting. I primi due livelli di dropout hanno un coefficiente di 0.2 mentre il terzo ha un coefficiente di 0.5. Anche in questo tool, ai tre layer convoluzionali, segue un fully-connected layer che prepara i dati all'output layer che, attraverso la funzione sigmoide, calcola la probabilità secondo il quale l'input fornito appartenga ad una tra le OCR disponibili.

A differenza di DeepSEA e Basset, il formato della sequenza in input è tridimensionale anziché a due dimensioni. In particolare la sequenza in input è di dimensione $M \times 4 \times (N + 1)$, dove M è la lunghezza della sequenza in input e 4 sono le basi azotate. Infine $N + 1$, ovvero la profondità della matrice tridimensionale indica le affinità specifiche che ha la sequenza di input con gli N fattori di trascrizione. A questo numero si aggiunge 1, che è lo strato della

codifica One-Hot della sequenza. In questo modo i TF associati alla sequenza di input aiutano a comprendere in maniera migliore gli effetti delle mutazioni nella sequenza. I TF sono stati ottenuti dal database JASPAR [84] e, mediante FIMO⁴ [85], sono stati identificati i motif più diffusi. Di questi, sono stati selezionati i primi N fattori di trascrizione più presenti da inserire nel dataset iniziale. Per motivi legati alle risorse disponibili, gli autori dell'articolo hanno scelto $N = 10$ siti di legame dei TF.

Il dataset che contiene sequenze di più specie è stato raccolto rispettivamente dai seguenti database:

- Le sequenze relative ai topi sono state ottenute dall'*NCBI Sequence Read Archive* (SRA) [86];
- I tratti di genoma umano sono stati ricavati da ENCODE;
- Le sequenze genomiche dei maiali sono state ottenute dal *Gene Expression Omnibus* (GEO) [87];
- Le porzioni di DNA relative a bovini e polli sono state scaricate dal sito dell'Università della California (*UC Davis*), nella sezione *Farm Clusters* [88].

Dopo aver raccolto tutti i dati necessari, in maniera del tutto analoga a DeepSEA, sono stati divisi in bin da 200 bp ed etichettati a seconda della OCR che rappresentavano: se più della metà della sequenza aveva una corrispondenza con una OCR, veniva impostata l'etichetta ad 1 per quella particolare regione della cromatina, altrimenti 0. In secondo luogo, sono stati centrati con sequenze lunghe 400 bp per avere sequenze di 1000 bp come input, chiamate anche *Open Chromatin Bin* (OCB). Di conseguenza, ciascun bin di input, una volta effettuato il One-Hot encoding è rappresentato da una matrice di dimensione $1000 \times 4 \times 11$.

Essendo DeepSATA un modello basato su DeepSEA, il modello è stato allenato attraverso il GD stocastico con momento per ottimizzare la funzione di binary cross entropy. In maniera del tutto analoga, il gradiente di questa funzione è stato calcolato mediante l'algoritmo della backpropagation

⁴Questo software identifica dei motif conosciuti all'interno della sequenza fornita in input.

5

Discussione

Dopo aver dettagliatamente descritto le caratteristiche dei tre strumenti bioinformatici nel Capitolo 4, in questo capitolo si paragoneranno i tre tool, sottolineando le proprietà comuni e gli aspetti per cui differiscono in modo da fornire un confronto oggettivo che permette di comprendere in maniera più approfondita le loro performance predittive.

DeepSEA, Basset e DeepSATA, hanno alcune caratteristiche in comune. I tre tool utilizzano lo stesso metodo di codifica della sequenza, il One-Hot encoding, che rende la sequenza luna M in una matrice $M \times 4$, mappando le basi azotate della sequenza in vettori. In questo modo la sequenza può essere processata correttamente dai kernel, che sono delle PWM che associano a ciascuna base delle sequenza un peso (o probabilità di occorrenza) in una particolare posizione (Figura 4.1). Infine i tre modelli sono stati allenati attraverso il SGD cercando di ottimizzare la stessa funzione obiettivo (cost function): la binary cross entropy (BCE).

Nonostante le similarità descritte, i tre strumenti presentano diverse differenze, soprattutto nella struttura della rete e nel training dataset scelto: la Tabella 5.1 riassume le caratteristiche principali. *DeepSEA* è composto da tre livelli convoluzionali, ciascuno dei quali viene seguito sempre da un livello ReLU e da un livello max-pooling, utilizzato per estrarre le feature dominanti processate dalla convoluzione. I tre livelli convoluzionali contengono rispettivamente 320, 480 e 960 filtri. In seguito è presente un fully-connected layer che prepara la informazioni per essere valutate nell'output layer, formato da una sigmoide.

Anche *Basset* è composto da tre livelli convoluzionali ma in aggiunta al ReLU layer e al max-pooling layer viene inserito anche un livello di normalizzazione. Sono poi presenti tre livelli fully-connected che sono alternati da altri ReLU layer e dei dropout layer, per evitare l'overfitting. Anche in questo caso l'output è calcolato tramite una sigmoide.

Come DeepSEA e Basset, anche *DeepSATA* è composto da tre livelli di convoluzione, con rispettivamente 320, 480 e 960 kernel. Ad ogni layer convoluzionale segue un ReLU layer, un max-pooling layer ed un dropout layer, per prevenire il rischio di overfitting. Ai livelli convoluzionali segue un fully connected layer che processa le informazioni per l'output layer che applica una funzione sigmoide. Si osserva inoltre che in questo modello, a differenza degli

Tabella 5.1: Riassunto delle differenze tra i tre tool.

Modello	Struttura	Kernel	Input	Dataset
<i>DeepSEA</i>	Tre livelli convoluzionali — ciascuno seguito da un ReLU layer e da un max-pooling layer — e un fully-connected layer	320, 480, 960	1000×4	ENCODE, Roadmap Epigenomics
<i>Basset</i>	Tre livelli convoluzionali — ciascuno seguito da un normalization layer, un ReLU layer e da un max-pooling layer — e tre fully-connected layer — alternati da un ReLU layer ed un dropout layer	300, 200, 200	600×4	ENCODE, Roadmap Epigenomics
<i>DeepSATA</i>	Tre livelli convoluzionali — ciascuno seguito da un ReLU layer, da un max-pooling layer e da un dropout layer — e un fully-connected layer	320, 480, 960	$1000 \times 4 \times 11$	ENCODE, NCBI SRA, GEO, UC Davis

altri due, processa un input tridimensionale, volto a fornire più contesto per comprendere in maniera migliore gli eventuali effetti delle mutazioni sulla sequenza.

Gli autori dell'articolo che introduce DeepSATA, dopo aver illustrato il modello e le principali differenze con il suo predecessore DeepSEA, conducono un esperimento dove paragonano le capacità predittive di DeepSEA, Basset e DeepSATA. In particolare, dopo aver allenato i tre modelli sul dataset contenente sequenze di specie animali diverse, tra cui l'uomo, valutano le prestazioni dei modelli. Dai risultati del test (Tabella 5.2) si nota che le prestazioni predittive di DeepSATA, anche se non di molto, superano le prestazioni di DeepSEA e di Basset. In particolare si nota una maggiore differenza nelle sequenze genetiche dei maiali, dove il valore AUC/AUROC è superiore di più di cinque punti percentuale. Nelle altre specie invece, anche se non di tanto, DeepSATA ottiene comunque risultati migliori. Dalla Tabella 5.2 si nota che i valori AUROC di DeepSATA e DeepSEA differiscono di pochi millesimi tra loro, a differenza dei valori di Basset, che talvolta sono nettamente più bassi rispetto ai valori degli altri due modelli. In particolare sulle sequenze che riguardano il genoma umano si nota una differenza di circa quattro punti percentuale tra Basset e DeepSEA/DeepSATA. Di conseguenza, la tabella, oltre che ad indicare che DeepSATA abbia capacità predittive migliori rispetto agli altri due strumenti, suggerisce che il modello di DeepSEA riesce a riconoscere in maniera migliore le categorie rispetto a Basset. Le informazioni riportate sulla Tabella 5.2 sono anche rappresenta-

Tabella 5.2: Risultati AUC relativi ai confronti tra DeepSEA, Basset e DeepSATA.

Modello	Topi	Maiali	Bovini	Umani	Polli
<i>DeepSATA</i>	0.854	0.779	0.772	0.759	0.744
<i>DeepSEA</i>	0.796	0.775	0.769	0.755	0.736
<i>Basset</i>	0.778	0.719	0.768	0.717	0.722

te sotto forma di curve ROC nella Figura 5.1, dove sono indicate le performance predittive di ciascun modello a seconda delle specie animali in esame. Per ciascuna specie animale e per ciascuno dei tre tool, sono stati rappresentati i grafici delle loro prestazioni. In particolare in ciascun grafico sono presenti diverse curve ROC, ciascuna delle quali indica la capacità predittiva rispetto ad una precisa OCR. Per ciascuna delle curve è stato calcolato il valore AUC e poi calcolata la media per ciascuna specie, ottenendo i valori indicati nella Tabella 5.2.

Infine, dagli articoli emergono alcune caratteristiche computazionali dei tool. In particolare, per allenare DeepSEA è stata utilizzata la GPU *NVIDIA Tesla K20m*, ma non viene specificato il tempo impiegato. È però disponibile una *repository GitHub* con una implementazione non ufficiale dello stesso tool, usando la libreria Python TensorFlow. Nella descrizione di questa repository viene indicato che per allenare completamente il tool da zero con una GTX 1070

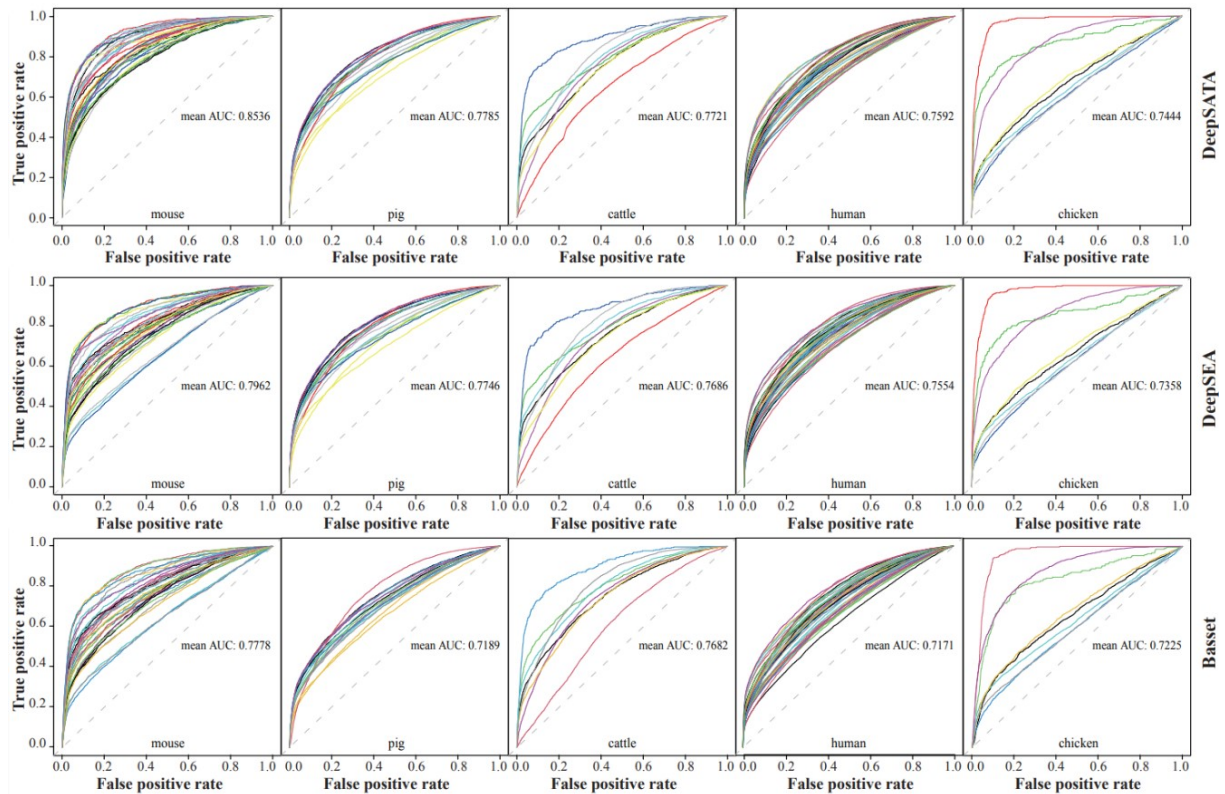


Figura 5.1: Confronto delle prestazioni predittive dei tre tool su diverse specie animali attraverso le curve ROC, mostrate per ogni OCR, e dai valori medi di AUROC [18].

sono stati necessari circa 10 giorni [89]. L'articolo di Basset, invece, descrive in modo più esemplificativo le sue caratteristiche. In particolare, l'allenamento del tool sulla stessa GPU usata per DeepSEA impiega 18 minuti, mentre, se allenato su un *MacBook 2.8-GHz Intel Core i7*, sono necessarie circa sei ore e mezza. DeepSATA, invece, non fornisce alcuna informazione sul modello di macchina utilizzato per l'allenamento o sul tempo impiegato, se non che è stato necessario limitare la dimensione dell'input a causa di vincoli di risorse.

In conclusione, vista la mancanza di informazioni complete da parte dei tre tool, in futuro si potrebbe effettuare un confronto sperimentale tra di essi, al fine di ottenere un paragone imparziale che fornisca in modo più dettagliato informazioni riguardanti la struttura, le richieste computazionali e le performance predittive dei tool.



Conclusioni

I tre strumenti bioinformatici discussi hanno dato un grande contributo alla comprensione degli effetti delle varianti non codificanti nel genoma umano. Per migliorare le prestazioni predittive e rendere questi strumenti ancora più precisi, si consiglia di allenare i modelli con dataset più ampi, così da approfondire meglio le ricadute funzionali delle mutazioni non codificanti. Le ricerche future si focalizzeranno sulla previsione delle interazioni tra enhancer e promoter e sul loro contributo all'espressione genica, offrendo nuove prospettive sugli effetti delle varianti non codificanti.

La progettazione di tool come questi, finalizzati allo studio delle varianti non codificanti, rimane di fondamentale importanza. Nonostante il progresso raggiunto grazie a questi strumenti, il campo delle mutazioni non codificanti è ancora poco esplorato dalla comunità scientifica e necessita quindi di un costante e continuo approfondimento e miglioramento. Le malattie legate a queste mutazioni sono diffuse e ancora poco comprese. È dunque essenziale proseguire le ricerche in questo ambito, nella speranza di fare nuove scoperte che possano arricchire la nostra comprensione e portare a soluzioni innovative.

Bibliografia

- [1] S. S. Sahu e G. Panda, «Identification of protein-coding regions in DNA sequences using a time-frequency filtering approach,» *Genomics, Proteomics and Bioinformatics*, vol. 9, n. 1-2, pp. 45–55, 2011.
- [2] T. D. Pollard, W. C. Earnshaw, J. Lippincott-Schwartz e G. Johnson, *Cell Biology E-Book: Cell Biology E-Book*. Elsevier Health Sciences, 2022.
- [3] F. Zhang e J. R. Lupski, «Non-coding genetic variants in human disease,» *Human molecular genetics*, vol. 24, n. R1, R102–R110, 2015.
- [4] J. French e S. Edwards, «The role of noncoding variants in heritable disease,» *Trends in Genetics*, vol. 36, n. 11, pp. 880–891, 2020.
- [5] H. Chial, «Mendelian genetics: patterns of inheritance and single-gene disorders,» *Nature Education*, vol. 1, n. 1, p. 63, 2008.
- [6] S. Pagni, J. D. Mills, A. Frankish, J. M. Mudge e S. M. Sisodiya, «Non-coding regulatory elements: Potential roles in disease and the case of epilepsy,» *Neuropathology and Applied Neurobiology*, vol. 48, n. 3, e12775, 2022.
- [7] A. Kapoor et al., «An enhancer polymorphism at the cardiomyocyte intercalated disc protein NOS1AP locus is a major regulator of the QT interval,» *The American Journal of Human Genetics*, vol. 94, n. 6, pp. 854–869, 2014.
- [8] E. Khurana, Y. Fu, D. Chakravarty, F. Demichelis, M. A. Rubin e M. Gerstein, «Role of non-coding sequence variants in cancer,» *Nature Reviews Genetics*, vol. 17, n. 2, pp. 93–108, 2016.
- [9] J. Tian et al., «Systematic functional interrogation of genes in GWAS loci identified ATF1 as a key driver in colorectal cancer modulated by a promoter-enhancer interaction,» *The American Journal of Human Genetics*, vol. 105, n. 1, pp. 29–47, 2019.
- [10] S. E. Bojesen et al., «Multiple independent variants at the TERT locus are associated with telomere length and risks of breast and ovarian cancer,» *Nature genetics*, vol. 45, n. 4, pp. 371–384, 2013.
- [11] K. Michailidou et al., «Association analysis identifies 65 new breast cancer risk loci,» *Nature*, vol. 551, n. 7678, pp. 92–94, 2017.
- [12] C. S. Pareek, R. Smoczynski e A. Tretyn, «Sequencing technologies and genome sequencing,» *Journal of applied genetics*, vol. 52, pp. 413–435, 2011.

- [13] S. K. Burley, H. M. Berman, G. J. Kleywegt, J. L. Markley, H. Nakamura e S. Velankar, «Protein Data Bank (PDB): the single global macromolecular structure archive,» *Protein crystallography: methods and protocols*, pp. 627–641, 2017.
- [14] N. M. Luscombe, D. Greenbaum e M. Gerstein, «What is bioinformatics? A proposed definition and overview of the field,» *Methods of information in medicine*, vol. 40, n. 04, pp. 346–358, 2001.
- [15] C. Caudai et al., «AI applications in functional genomics,» *Computational and Structural Biotechnology Journal*, vol. 19, pp. 5762–5790, 2021.
- [16] J. Zhou e O. G. Troyanskaya, «Predicting effects of noncoding variants with deep learning-based sequence model,» *Nature methods*, vol. 12, n. 10, pp. 931–934, 2015.
- [17] D. R. Kelley, J. Snoek e J. L. Rinn, «Basset: learning the regulatory code of the accessible genome with deep convolutional neural networks,» *Genome research*, vol. 26, n. 7, pp. 990–999, 2016.
- [18] W. Ma et al., «DeepSATA: A Deep Learning-Based Sequence Analyzer Incorporating the Transcription Factor Binding Affinity to Dissect the Effects of Non-Coding Genetic Variants,» *International Journal of Molecular Sciences*, vol. 24, n. 15, p. 12 023, 2023.
- [19] B. Alberts et al., *Essential cell biology*. Garland Science, 2015.
- [20] P. F. Chinnery e E. A. Schon, «Mitochondria,» *Journal of Neurology, Neurosurgery & Psychiatry*, vol. 74, n. 9, pp. 1188–1199, 2003.
- [21] H. M. McBride, M. Neuspiel e S. Wasiak, «Mitochondria: more than just a powerhouse,» *Current biology*, vol. 16, n. 14, R551–R560, 2006.
- [22] G. K. Voeltz, M. M. Rolls e T. A. Rapoport, «Structural organization of the endoplasmic reticulum,» *EMBO reports*, vol. 3, n. 10, pp. 944–950, 2002.
- [23] A. Ballabio, «The awesome lysosome,» *EMBO molecular medicine*, vol. 8, n. 2, pp. 73–76, 2016.
- [24] C. Yang e X. Wang, «Lysosome biogenesis: Regulation and functions,» *The Journal of cell biology*, vol. 220, n. 6, 2021.
- [25] E. C. Dell’Angelica, C. Mullins, S. Caplan e J. S. Bonifacino, «Lysosome-related organelles,» *The FASEB Journal*, vol. 14, n. 10, pp. 1265–1278, 2000.
- [26] M. Islinger, S. Grille, H. D. Fahimi e M. Schrader, «The peroxisome: an update on mysteries,» *Histochemistry and cell biology*, vol. 137, pp. 547–574, 2012.
- [27] National Human Genome Research Institute, *Deoxyribonucleic acid (DNA) Image*, <https://www.genome.gov/genetics-glossary/Deoxyribonucleic-Acid>, 2024.
- [28] C. Fonseca Guerra, F. M. Bickelhaupt, J. G. Snijders e E. J. Baerends, «Hydrogen bonding in DNA base pairs: reconciliation of theory and experiment,» *Journal of the American Chemical Society*, vol. 122, n. 17, pp. 4117–4128, 2000.

- [29] A. Jansen e K. J. Verstrepen, «Nucleosome positioning in *Saccharomyces cerevisiae*,» *Microbiology and molecular biology reviews*, vol. 75, n. 2, pp. 301–320, 2011.
- [30] G. Zheng, *The packaging of DNA in chromatin*. Rutgers The State University of New Jersey, School of Graduate Studies, 2010.
- [31] National Human Genome Research Institute, *Chromosome Image*, <https://www.genome.gov/genetics-glossary/Chromosome>, 2024.
- [32] M. B. Gerstein et al., «What is a gene, post-ENCODE? History and updated definition,» *Genome research*, vol. 17, n. 6, pp. 669–681, 2007.
- [33] R. J. White, *Gene transcription: mechanisms and control*. John Wiley & Sons, 2009.
- [34] B. Alberts, A. Johnson, J. Lewis, M. Raff, K. Roberts e P. Walter, «From DNA to RNA,» in *Molecular Biology of the Cell. 4th edition*, Garland Science, 2002.
- [35] P. Cramer, «Organization and regulation of gene transcription,» *Nature*, vol. 573, n. 7772, pp. 45–54, 2019.
- [36] National Human Genome Research Institute, *Transcription Image*, <https://www.genome.gov/genetics-glossary/Transcription>, 2024.
- [37] A. Philips e T. Cooper*, «RNA processing and human disease,» *Cellular and Molecular Life Sciences CMLS*, vol. 57, pp. 235–249, 2000.
- [38] S. Hocine, R. H. Singer e D. Grünwald, «RNA processing and export,» *Cold Spring Harbor perspectives in biology*, vol. 2, n. 12, a000752, 2010.
- [39] M. Livingstone, E. Atas, A. Meller e N. Sonenberg, «Mechanisms governing the control of mRNA translation,» *Physical biology*, vol. 7, n. 2, p. 021 001, 2010.
- [40] V. Ramakrishnan, «Ribosome structure and the mechanism of translation,» *Cell*, vol. 108, n. 4, pp. 557–572, 2002.
- [41] J. Lemonnier, N. Lemonnier, S. Pascolo e C. Pichon, «The Marathon of the Messenger,»
- [42] National Human Genome Research Institute, *Translation Image*, <https://www.genome.gov/genetics-glossary/Translation>, 2024.
- [43] G. E. Schulz e R. H. Schirmer, *Principles of protein structure*. Springer Science & Business Media, 2013.
- [44] R. M. Bavle, «Mitosis at a glance,» *Journal of Oral and Maxillofacial Pathology*, vol. 18, n. Suppl 1, S2–S5, 2014.
- [45] C. E. Walczak, S. Cai e A. Khodjakov, «Mechanisms of chromosome behaviour during mitosis,» *Nature reviews Molecular cell biology*, vol. 11, n. 2, pp. 91–102, 2010.
- [46] X. Li, F. Yang e B. Rubinsky, «A theoretical study on the biophysical mechanisms by which tumor treating fields affect tumor cells during mitosis,» *IEEE Transactions on Biomedical Engineering*, vol. 67, n. 9, pp. 2594–2602, 2020.

- [47] M. Sullivan e D. O. Morgan, «Finishing mitosis, one step at a time,» *Nature reviews Molecular cell biology*, vol. 8, n. 11, pp. 894–903, 2007.
- [48] National Human Genome Research Institute, *Mitosis Image*, <https://www.genome.gov/genetics-glossary/Mitosis>, 2024.
- [49] R. A. Laskey, M. P. Fairman e J. J. Blow, «S phase of the cell cycle,» *Science*, vol. 246, n. 4930, pp. 609–614, 1989.
- [50] S. P. Bell e A. Dutta, «DNA replication in eukaryotic cells,» *Annual review of biochemistry*, vol. 71, n. 1, pp. 333–374, 2002.
- [51] A. Dutta e S. P. Bell, «Initiation of DNA replication in eukaryotic cells,» *Annual review of cell and developmental biology*, vol. 13, n. 1, pp. 293–332, 1997.
- [52] «Chapter 42 - S Phase and DNA Replication,» in *Cell Biology (Third Edition)*, T. D. Pollard, W. C. Earnshaw, J. Lippincott-Schwartz e G. T. Johnson, cur., Third Edition, Elsevier, 2017, pp. 727–741, ISBN: 978-0-323-34126-4. DOI: <https://doi.org/10.1016/B978-0-323-34126-4.00042-6>. indirizzo: <https://www.sciencedirect.com/science/article/pii/B9780323341264000426>.
- [53] National Human Genome Research Institute, *DNA Replication Image*, <https://www.genome.gov/genetics-glossary/DNA-Replication>, 2024.
- [54] P. M. Visscher, M. A. Brown, M. I. McCarthy e J. Yang, «Five years of GWAS discovery,» *The American Journal of Human Genetics*, vol. 90, n. 1, pp. 7–24, 2012.
- [55] M. Z. Ludwig, «Functional evolution of noncoding DNA,» *Current opinion in genetics & development*, vol. 12, n. 6, pp. 634–639, 2002.
- [56] V. B. Kaiser e C. A. Semple, «When TADs go bad: chromatin structure and nuclear organisation in human disease,» *F1000Research*, vol. 6, 2017.
- [57] M. Schipper e D. Posthuma, «Demystifying non-coding GWAS variants: an overview of computational tools and methods,» *Human molecular genetics*, vol. 31, n. R1, R73–R83, 2022.
- [58] E. G. Peña-Martínez e J. A. Rodríguez-Martínez, «Decoding Non-coding Variants: Recent Approaches to Studying Their Role in Gene Regulation and Human Diseases,» *Frontiers in bioscience (Scholar edition)*, vol. 16, n. 1, p. 4, 2024.
- [59] W. S. McCulloch e W. Pitts, «A logical calculus of the ideas immanent in nervous activity,» *The bulletin of mathematical biophysics*, vol. 5, pp. 115–133, 1943.
- [60] A. M. Turing, *Computing machinery and intelligence*. Springer, 2009.
- [61] F. Rosenblatt, «The perceptron: a probabilistic model for information storage and organization in the brain.,» *Psychological review*, vol. 65, n. 6, p. 386, 1958.
- [62] M. Minsky e S. A. Papert, *Perceptrons, reissue of the 1988 expanded edition with a new foreword by Léon Bottou: an introduction to computational geometry*. MIT press, 2017.

- [63] D. E. Rumelhart, G. E. Hinton e R. J. Williams, «Learning representations by back-propagating errors,» *nature*, vol. 323, n. 6088, pp. 533–536, 1986.
- [64] M. Flasiński, *Introduction to artificial intelligence*. Springer, 2016.
- [65] N. Muthukrishnan, F. Maleki, K. Ovens, C. Reinhold, B. Forghani, R. Forghani et al., «Brief history of artificial intelligence,» *Neuroimaging Clinics of North America*, vol. 30, n. 4, pp. 393–399, 2020.
- [66] M. A. Nielsen, *Neural networks and deep learning*. Determination press San Francisco, CA, USA, 2015, vol. 25.
- [67] J. Lu, «Gradient descent, stochastic optimization, and other tales,» *arXiv preprint arXiv:2205.00832*, 2022.
- [68] M. Andrychowicz et al., «Learning to learn by gradient descent by gradient descent,» *Advances in neural information processing systems*, vol. 29, 2016.
- [69] I. Neutelings, *Neural Netowrk with coefficients, shifted*, Licensed under Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0)., 2022. indiriz-zo: https://tikz.net/neural_networks/.
- [70] R. Rojas e R. Rojas, «The backpropagation algorithm,» *Neural networks: a systematic introduction*, pp. 149–182, 1996.
- [71] S. Narkhede, «Understanding auc-roc curve,» *Towards data science*, vol. 26, n. 1, pp. 220–227, 2018.
- [72] J. Huang e C. X. Ling, «Using AUC and accuracy in evaluating learning algorithms,» *IEEE Transactions on knowledge and Data Engineering*, vol. 17, n. 3, pp. 299–310, 2005.
- [73] Y. LeCun, L. Bottou, Y. Bengio e P. Haffner, «Gradient-based learning applied to document recognition,» *Proceedings of the IEEE*, vol. 86, n. 11, pp. 2278–2324, 1998.
- [74] C. C. Aggarwal et al., *Neural networks and deep learning*. Springer, 2018, vol. 10.
- [75] J. Riebesell, *Convolution Operator*, Licensed under Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0)., 2022. indirizzo: <https://tikz.net/conv2d/>.
- [76] J. Wu, «Introduction to convolutional neural networks,» *National Key Lab for Novel Software Technology. Nanjing University. China*, vol. 5, n. 23, p. 495, 2017.
- [77] K. O’Shea, «An introduction to convolutional neural networks,» *arXiv preprint arXiv:1511.08458*, 2015.
- [78] A. S. Shamsaldin, P. Fattah, T. A. Rashid e N. K. Al-Salihi, «A study of the applications of convolutional neural networks,» *J. Sci. Eng.*, vol. 3, pp. 31–39, 2019.
- [79] Z. Li, F. Liu, W. Yang, S. Peng e J. Zhou, «A survey of convolutional neural networks: analysis, applications, and prospects,» *IEEE transactions on neural networks and learning systems*, vol. 33, n. 12, pp. 6999–7019, 2021.

- [80] D. Bhatt et al., «CNN variants for computer vision: History, architecture, application, challenges and future scope,» *Electronics*, vol. 10, n. 20, p. 2470, 2021.
- [81] S. Min, B. Lee e S. Yoon, «Deep learning in bioinformatics,» *Briefings in bioinformatics*, vol. 18, n. 5, pp. 851–869, 2017.
- [82] *ENCODE Project*, <https://www.encodeproject.org>.
- [83] *Roadmap Epigenomics*, <http://www.roadmapepigenomics.org>.
- [84] *JASPAR*, <https://jaspar.elixir.no>.
- [85] C. E. Grant, T. L. Bailey e W. S. Noble, «FIMO: scanning for occurrences of a given motif,» *Bioinformatics*, vol. 27, n. 7, pp. 1017–1018, 2011.
- [86] *NCBI Sequence Read Archive*, <https://www.ncbi.nlm.nih.gov/sra>.
- [87] *NCBI Gene Expression Omnibus*, <https://www.ncbi.nlm.nih.gov/geo/>.
- [88] *UC Davis — Farm Clusters*, https://farm.cse.ucdavis.edu/~ckern/Nature_Communications_2020/Peak_Calls/.
- [89] J. Wu, *DeepSEA in TensorFlow*, <https://github.com/jimmyhwu/deepsea>.

Ringraziamenti

Con queste righe desidero esprimere la mia riconoscenza alla mia relatrice, la Professoressa Pizzi, per la sua disponibilità nell'aiutarmi durante la stesura della tesi e per le sue risposte tempestive ed esaustive.

Un ringraziamento va anche all'Università degli Studi di Padova per avermi offerto l'opportunità di partecipare al programma Erasmus+, un'esperienza che ha arricchito significativamente il mio percorso accademico e personale.

Infine, vorrei esprimere la mia gratitudine alla mia famiglia e a tutte le persone che mi hanno sostenuto e accompagnato in questo straordinario viaggio.